



Systems Software

Course Code	CSE 2103
Course Name	Systems Software
Credits	5
Pre-Requisites	NA
L-T-P-C	3-1-2-5
COURSE OBJECTIVE	
Upon successful completion of this course, students will be able to understand: • Skills that will enable them to construct software of high quality – software that is reliable, and that is reasonably easy to understand, modify and maintain. • The detailed structure of any operating system along with its services. • The execution of shell commands and scripts in Unix/Linux Platform. • About various system calls and the implication of the fork() system call in Unix/Linux. • The basics of Assembler and Assembly language in any of the platform i.e. NASM, TASM etc.	

LECTUREs WITH BREAKUP	No. of Lectures
Operating System: Structure of a Computer System, Define Operating System, Types of Operating Systems, Operating System and Architecture, Objectives of Operating Systems, Functions of the Operating System, How to Choose the Operating System.	8
Shell Programming: Introduction to Shell Programming using Linux, Variable, echo, User input, Decision Making: if, if else, if elif else, case, Looping: for, while, do while and functions	7
Operating System Services Processes, Process structure, Process Scheduling, Scheduling algorithms, Process Synchronization & Deadlocks, Thread Management, Memory Management, System Calls, File System.	7
Compiler and Linker: Von Neumann machine architecture, Compilers, Interpreters and Linkers-Memory allocation, Code Optimization, Linking and Relocation Concepts	8
Assembler Basic function, Machine dependent and independent assembler features, Assembler design options: Two-pass, one-pass and multi-pass assembler design.	8

COURSE OUTCOMES
On completion of the course the student should be able to;
• Understand the Structure of a Computer System and applications. • Understand the Shell Programming concepts using linux. • Understand different Operating System Services. • Identify and understand concept of compilers and linker • Understand the basics of Assembler.

Reference Material-

1. D. M. Dhamdhare, 1999, Systems Programming and Operating Systems, Second Revised Edition, Tata McGraw-Hill, New Delhi.
2. ABRAHAM SILBERSCHATZ, PETER GALVIN, GREG GAGNE, Operating System Concepts, from Eighth Edition.
3. L. L. Beck, 1996, System Software An Introduction to System Programming, from Third edition, Addison-Wesley.



By

Dr. Jagadish S Jakati

Assistant Professor, SCSEA

D. Y. Patil International

University, Akurdi, Pune, 411044,

Maharashtra, India

Email: jagadish.jakati@dypiu.ac.in

Website: www.dypiu.ac.in

**D Y PATIL
INTERNATIONAL
UNIVERSITY
AKURDI PUNE**

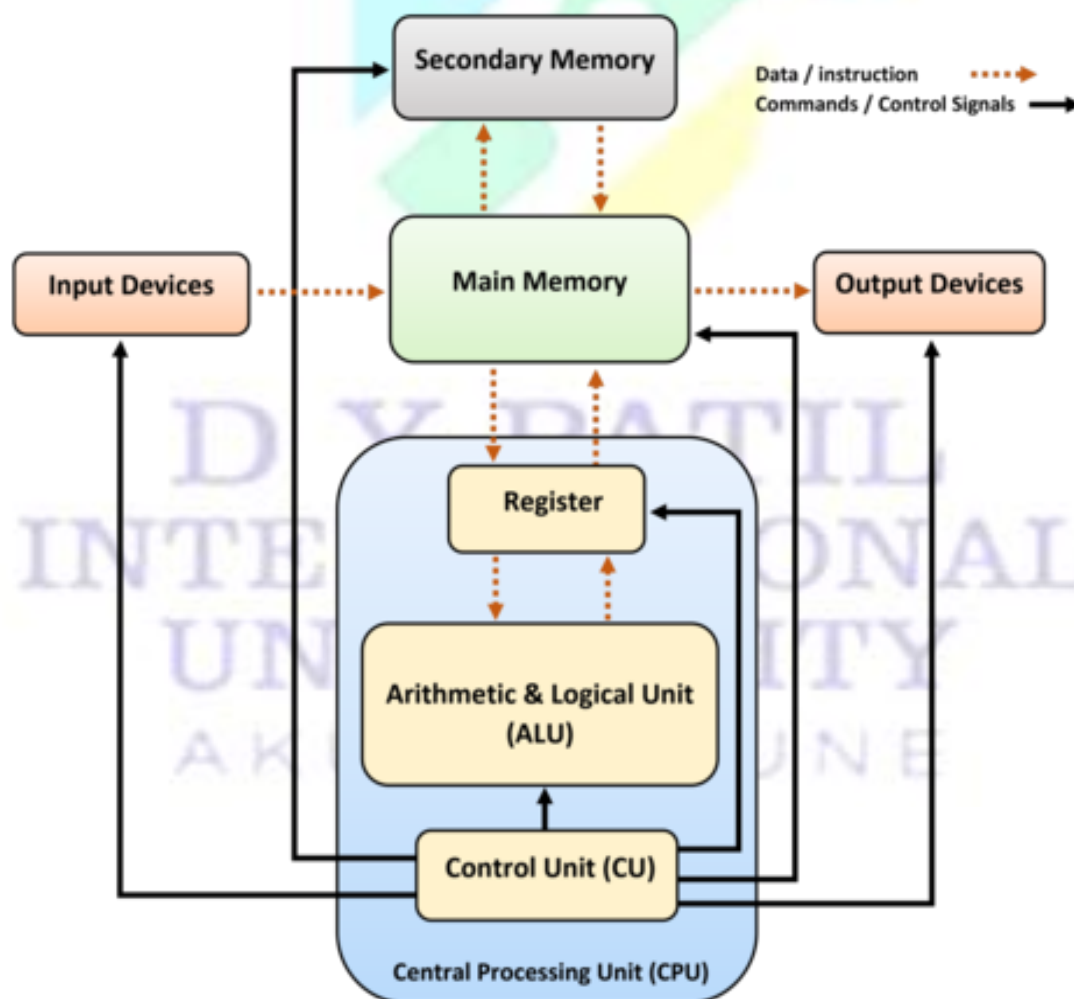
Module-01**RBT Level-L1, L2****Operating System:**

Structure of a Computer System, Define Operating System, Types of Operating Systems, Operating System and Architecture, Objectives of Operating Systems, Functions of the Operating System, How to Choose the Operating System.

Structure of a Computer System

Computer structure is the way that each component is arranged so that communication is possible.

The structure of a computer is simple, and that can be represented in below diagram –



Parts of Computer

The computer structure in general, divided into five main parts, which are as follows –

- Input devices
- Output devices
- I / O Ports
- CPU and
- Memory

Input Device

An input device is a computer hardware which handles input receiving from outside the system. The main function of the input device is to enter data or give commands by the user into the system.

For example, the input devices are keyboards, mouse, joysticks, scanners, touchpad and many more.



KEYBOARD



SCANNER



JOYSTICK



MOUSE



LIGHT PEN



WEBCAM



MICROPHONE

Output devices

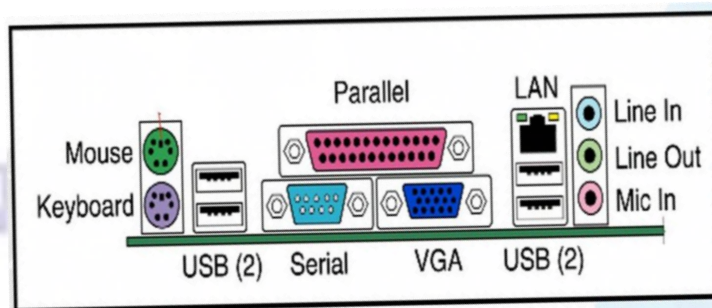
Output devices are computer hardware devices which handle output activities. The function of the output device is to display the results of data processing, as soft copy, hard copy, voice, or video.

For example, the output devices are monitors, printers, speakers, projectors.



I/O Ports

I / O Ports are called connectors which are under the control of the processor helpful in handling input and output activities. Or in other words, this port is used to connect any device that comes from outside the computer, like a printer, scanner, mouse, or USB. In a computer system this port is present on the back of the CPU while on the laptop the ports are located on the right and left side.



CPU

The Central Processing unit is computer hardware called the brain of the computer system. Because, the performance of a computer is determined by the technology and quality of the CPU used. Generally, there are three types of components in the CPU, which are given below –

- Control unit
- Register
- ALU (Arithmetic Logic Unit)

Control unit

A control unit controls the operations of all parts of the computer but it does not carry out any data processing operations.

Functions of the Control Unit

- It coordinates the sequence of data movements into, out of, and between a processor's many sub-units.
- It interprets instructions.
- It controls data flow inside the processor.
- It receives external instructions or commands to which it converts to sequence of control signals.
- It controls many execution units(i.e.ALU , data buffers and registers) contained within a CPU.
- It also handles multiple tasks, such as fetching, decoding, execution handling and storing results.

Register

Registers are a type of computer memory used to quickly accept, store, and transfer data and instructions that are being used immediately by the CPU.

A processor register may hold an instruction, a storage address, or any data (such as bit sequence or individual characters).

The computer needs processor registers for manipulating data and a register for holding a memory address. The register holding the memory location is used to calculate the address of the next instruction after the execution of the current instruction is completed.

Following is the list of some of the most common registers used in a basic computer:

Register	Symbol	Number of bits	Function
Data register	DR	16	Holds memory operand
Address register	AR	12	Holds address for the memory
Accumulator	AC	16	Processor register
Instruction register	IR	16	Holds instruction code
Program counter	PC	12	Holds address of the instruction

Temporary register	TR	16	Holds temporary data
Input register	INPR	8	Carries input character
Output register	OUTR	8	Carries output character

ALU (Arithmetic Logic Unit)

The first ALU (Arithmetic Logic Unit) was indeed the INTEL 74181, which was implemented as part of the 7400 series TTL (Transistor-Transistor Logic) integrated circuits. It was released by Intel in 1970.

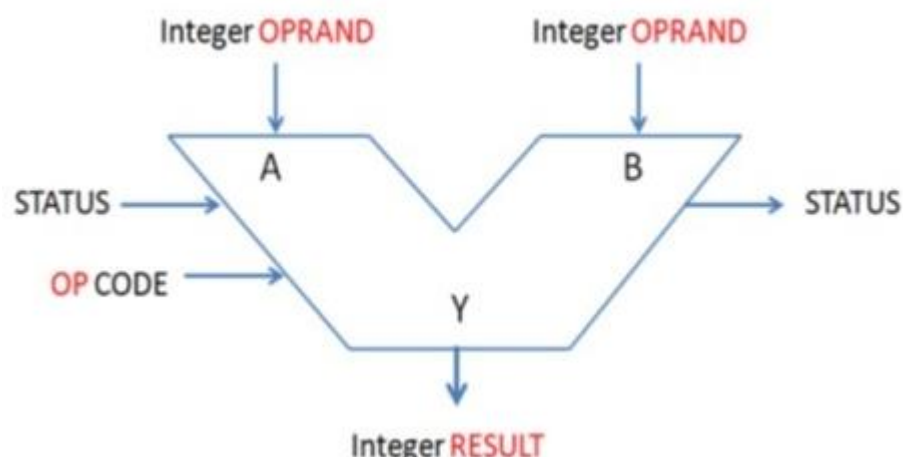
ALU is a digital circuit that performs all arithmetic and logic operations based on the instructions given.

The operations performed by ALU are:

Logical Operations: The logical operations consist of NOR, NOT, AND, NAND, OR, XOR, and more.

Bit-Shifting Operations: It is responsible for displacement in the locations of the bits to the by right or left by a certain number of places that are known as a multiplication operation.

Arithmetic Operations: Although it performs multiplication and division, this refers to bit addition and subtraction. But multiplication and division operations are more costly to make. In the place of multiplication, addition can be used as a substitute and subtraction for division.



Working of ALU:

Initially control unit decode the operation code (OPCODE) part of the machine instruction.

The Control unit then direct the arithmetic logic unit to perform the specific operation as per the operation code.

The ALU operates on the data (Operand) part of the machine instruction and processed the data is stored in the memory unit.

Define Operating System.

An **operating system (OS)** is system software that manages computer hardware and software resources, and provides common services for computer programs.

OR

An **Operating System** is a System software that manages all the resources of the computing device.

- Acts as an interface between the software and different parts of the computer or the computer hardware.
- Manages the overall resources and operations of the computer.
- Controls and monitors the execution of all other programs that reside in the computer, which also includes application programs and other system software of the computer.

Functions of the Operating System

Resource Management: The operating system manages and allocates memory, CPU time, and other hardware resources among the various programs and processes running on the computer.

Process Management: The operating system is responsible for starting, stopping, and managing processes and programs. It also controls the scheduling of processes and allocates resources to them.

Memory Management: The operating system manages the computer's primary memory and provides mechanisms for optimizing memory usage.

Security: The operating system provides a secure environment for the user, applications, and data by implementing security policies and mechanisms such as access controls and encryption.

Job Accounting: It keeps track of time and resources used by various jobs or users.

File Management: The operating system is responsible for organizing and managing the file system, including the creation, deletion, and manipulation of files and directories.

Device Management: The operating system manages input/output devices such as printers, keyboards, mice, and displays. It provides the necessary drivers and interfaces to enable communication between the devices and the computer.

Networking: The operating system provides networking capabilities such as establishing and managing network connections, handling network protocols, and sharing resources such as printers and files over a network.

User Interface: The operating system provides a user interface that enables users to interact with the computer system. This can be a Graphical User Interface (GUI), a Command-Line Interface (CLI), or a combination of both.

Backup and Recovery: The operating system provides mechanisms for backing up data and recovering it in case of system failures, errors, or disasters.

Virtualization: The operating system provides virtualization capabilities that allow multiple operating systems or applications to run on a single physical machine. This can enable efficient use of resources and flexibility in managing workloads.

Performance Monitoring: The operating system provides tools for monitoring and optimizing system performance, including identifying bottlenecks, optimizing resource usage, and analyzing system logs and metrics.

Time-Sharing: The operating system enables multiple users to share a computer system and its resources simultaneously by providing time-sharing mechanisms that allocate resources fairly and efficiently.

System Calls: The operating system provides a set of system calls that enable applications to interact with the operating system and access its resources. System calls provide a standardized interface between applications and the operating system, enabling portability and compatibility across different hardware and software platforms.

Error-detecting Aids: These contain methods that include the error messages, and other debugging and error-detecting methods.

Objectives of Operating Systems

Convenient to use: One of the objectives is to make the computer system more convenient to use in an efficient manner.

User Friendly: To make the computer system more interactive with a more convenient interface for the users.

Easy Access: To provide easy access to users for using resources by acting as an intermediary between the hardware and its users.

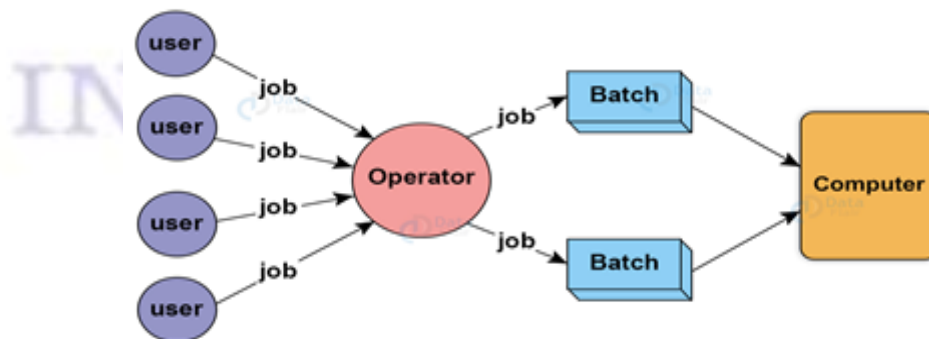
Management of Resources: For managing the resources of a computer in a better and faster way.

Controls and Monitoring: By keeping track of who is using which resource, granting resource requests, and mediating conflicting requests from different programs and users.

Fair Sharing of Resources: Providing efficient and fair sharing of resources between the users and programs.

Types of Operating Systems

Batch Operating System: A Batch Operating System is a type of operating system that does not interact with the computer directly. There is an operator who takes similar jobs having the same requirements and groups them into batches.



Batch operating system is a type of operating system which does not communicate or interact with the computer directly. It is helpful when computer processes are very lengthy and time-consuming. It speeds up the process by combining together similar types of jobs and runs them as a group.

There is an operator in this kind of operating system which is used to club similar jobs having the same requirement and is responsible to sort jobs with similar needs. **Eg: Bank System, Payroll System, etc**

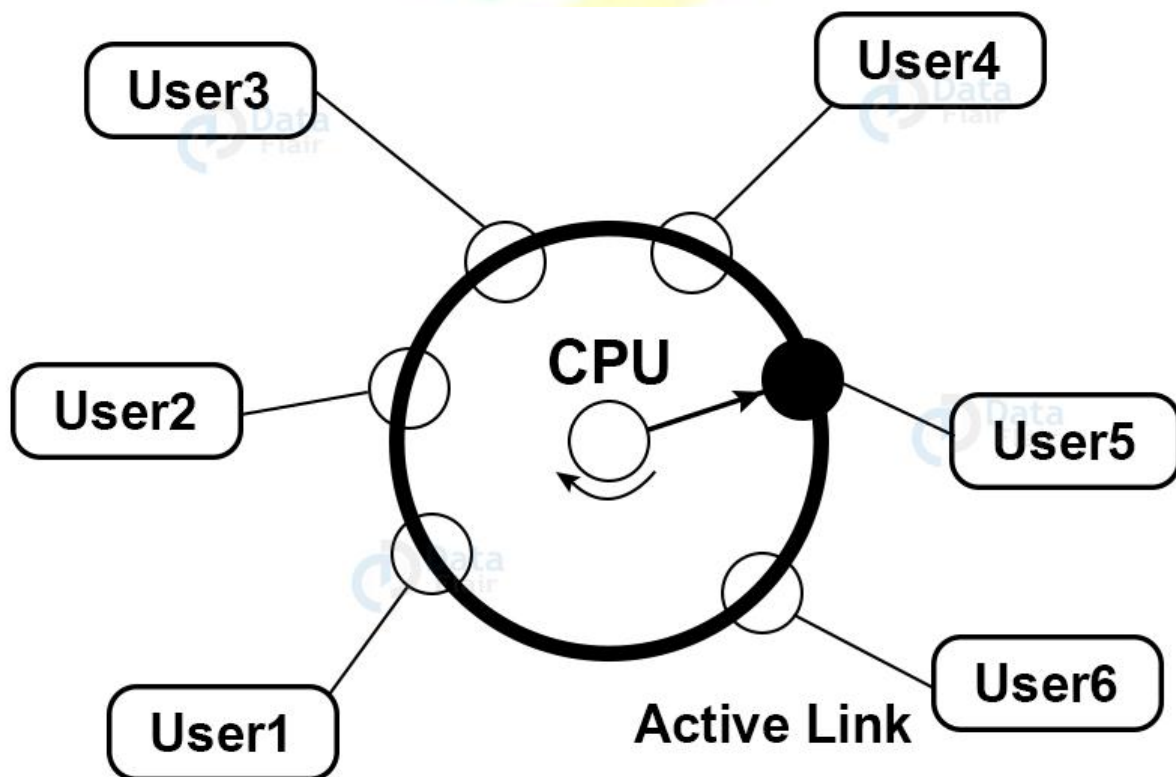
Advantages of Batch Operating System

- Multiple users can share the batch systems.
- The idle time for the batch system is very less.
- It is easy to manage large work repeatedly in batch systems.

Disadvantages of Batch Operating System

- CPU is not used efficiently. When the current process is doing IO, CPU is free and could be utilized by other processes waiting.
- The other jobs will have to wait for an unknown time if any job fails.
- In batch operating system, average response time increases as all processes are processed one by one.

Time-sharing Operating System: Time-sharing Operating System is a type of operating system that allows many users to share computer resources (maximum utilization of the resources).



This type of operating system enables people who are in two different shells to use the computer system at the same time. The processing time is termed time sharing as it is shared among multiple users.

Each task is allotted some time (known as Quantum) to execute its work. After the allotted time interval is over the operating system switches over to the next task

Eg: UNIX, Multics

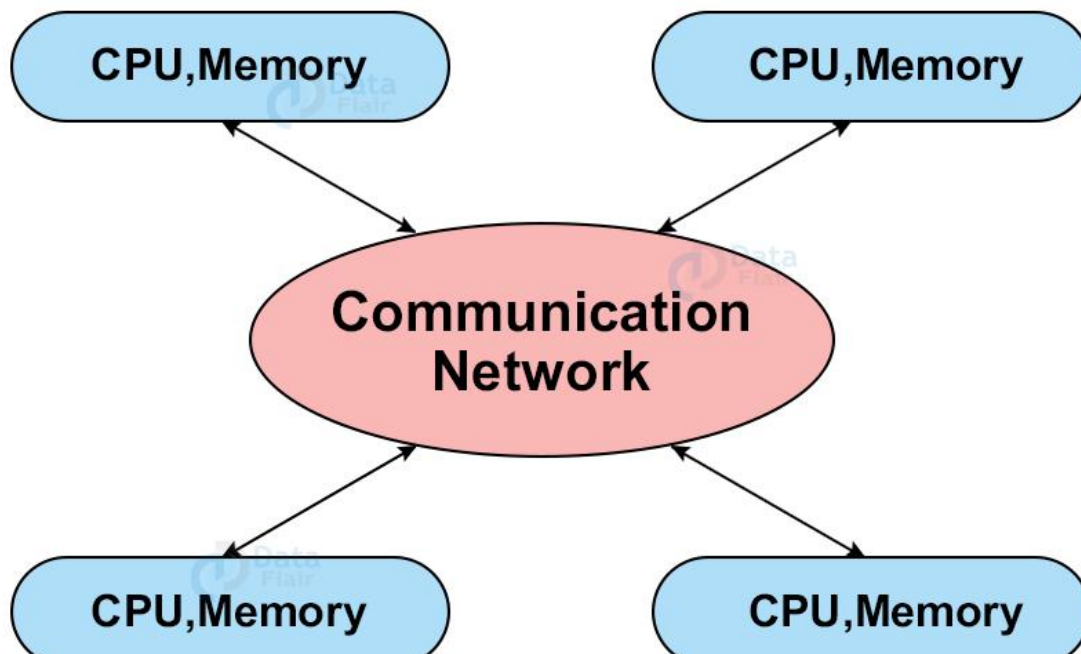
Advantages of Time-Sharing OS

- Each task gets an equal opportunity.
- Fewer chances of duplication of software.
- CPU idle time can be reduced.
- Resource Sharing: Time-sharing systems allow multiple users to share hardware resources such as the CPU, memory, and peripherals, reducing the cost of hardware and increasing efficiency.
- Improved Productivity: Time-sharing allows users to work concurrently, thereby reducing the waiting time for their turn to use the computer. This increased productivity translates to more work getting done in less time.
- Improved User Experience: Time-sharing provides an interactive environment that allows users to communicate with the computer in real time, providing a better user experience than batch processing.

Disadvantages of Time-Sharing OS

- Reliability problem.
- One must have to take care of the security and integrity of user programs and data.
- Data communication problem.
- High Overhead: Time-sharing systems have a higher overhead than other operating systems due to the need for scheduling, context switching, and other overheads that come with supporting multiple users.
- Complexity: Time-sharing systems are complex and require advanced software to manage multiple users simultaneously. This complexity increases the chance of bugs and errors.
- Security Risks: With multiple users sharing resources, the risk of security breaches increases. Time-sharing systems require careful management of user access, authentication, and authorization to ensure the security of data and software.

Distributed Operating System: Distributed Operating System is a type of operating system that manages a group of different computers and makes appear to be a single computer. These operating systems are designed to operate on a network of computers. They allow multiple users to access shared resources and communicate with each other over the network. **Examples include Microsoft Windows Server and various distributions of Linux designed for servers.**



In distributed operating systems communication networks are used, because of which various interconnected computers are able to communicate with each other. It is basically a system software over a collection of communicating and physically separate computation nodes. These systems contribute to the capability of operating systems to manage data security applications and other functions by running on a server. Multiple Central Processes are used by the system to serve multiple users and real time. It enables resource sharing and is fault tolerant.

Eg: LOCUS, etc.

Advantages of Distributed Operating System

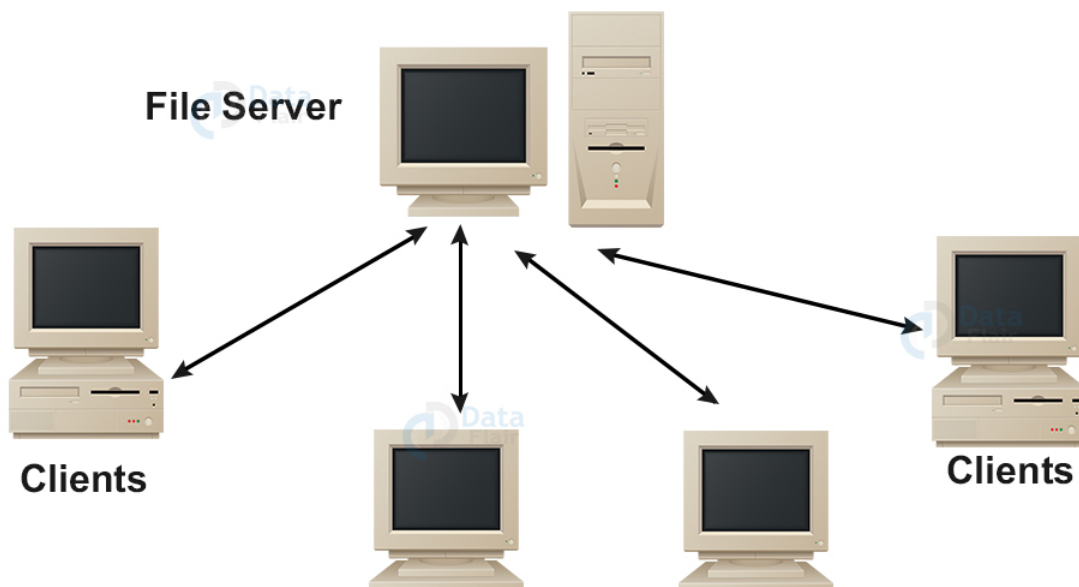
- Failure of one will not affect the other network communication, as all systems are independent of each other.
- Electronic mail increases the data exchange speed.
- Since resources are being shared, computation is highly fast and durable.
- Load on host computer reduces.

- These systems are easily scalable as many systems can be easily added to the network.
- Delay in data processing reduces.

Disadvantages of Distributed Operating System

- Failure of the main network will stop the entire communication.
- To establish distributed systems the language is used not well-defined yet.
- These types of systems are not readily available as they are very expensive. Not only that the underlying software is highly complex and not understood well yet.

Network Operating System: Network Operating System is a type of operating system that runs on a server and provides the capability to manage data, users, groups, security, applications, and other networking functions.



Network Operating System runs on a server and makes the system capable of managing data and other networking functions. It helps to connect various independent computers over a network. NOS is a specialized operating system for a network device that manages multiple requests for inputs parallelly and provides the necessary security in a multiuser environment NOS may also offer directory services and an email system or network management and multiprotocol routing capabilities.

Eg: Mac OS X, Novell NetWare, Microsoft Windows Server 2003

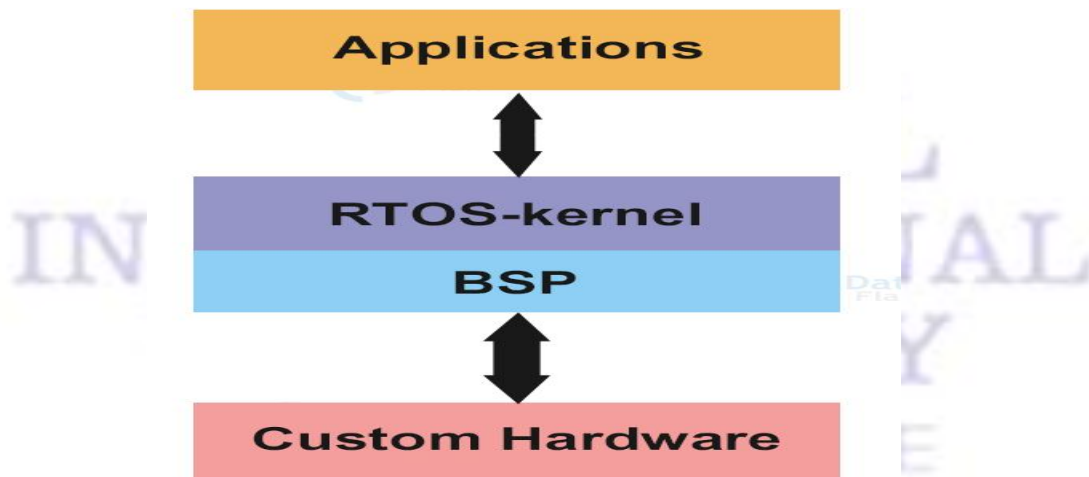
Advantages of Network Operating System

- Highly stable centralized servers.
- Security concerns are handled through servers.
- New technologies and hardware up-gradation are easily integrated into the system.
- Server access is possible remotely from different locations and types of systems.

Disadvantages of Network Operating System

- Servers are costly.
- User has to depend on a central location for most operations.
- Maintenance and updates are required regularly.

Real-time Operating System: Real-time Operating System is a type of operating system that serves a real-time system and the time interval required to process and respond to inputs is very small. These operating systems are designed to respond to events in real time. They are used in applications that require quick and deterministic responses, such as embedded systems, industrial control systems, and robotics.



Real-time operating systems serve the real-time systems which means that the system is subjected to real time where the response should be guaranteed within a specific timing constraint and so in this type of operating system the time constant is very less.

Eg: Industrial control systems, Scientific experiments, Air traffic control systems, etc.

Advantages of RTOS

- **Maximum Consumption:** Maximum utilization of devices and systems, thus more output from all the resources.
- **Task Shifting:** The time assigned for shifting tasks in these systems is very less. For example, in older systems, it takes about 10 microseconds in shifting from one task to another, and in the latest systems, it takes 3 microseconds.
- **Focus on Application:** Focus on running applications and less importance on applications that are in the queue.
- **Real-time operating system in the embedded system:** Since the size of programs is small, RTOS can also be used in embedded systems like in transport and others.
- **Error Free:** These types of systems are error-free.
- **Memory Allocation:** Memory allocation is best managed in these types of systems.

Disadvantages of RTOS

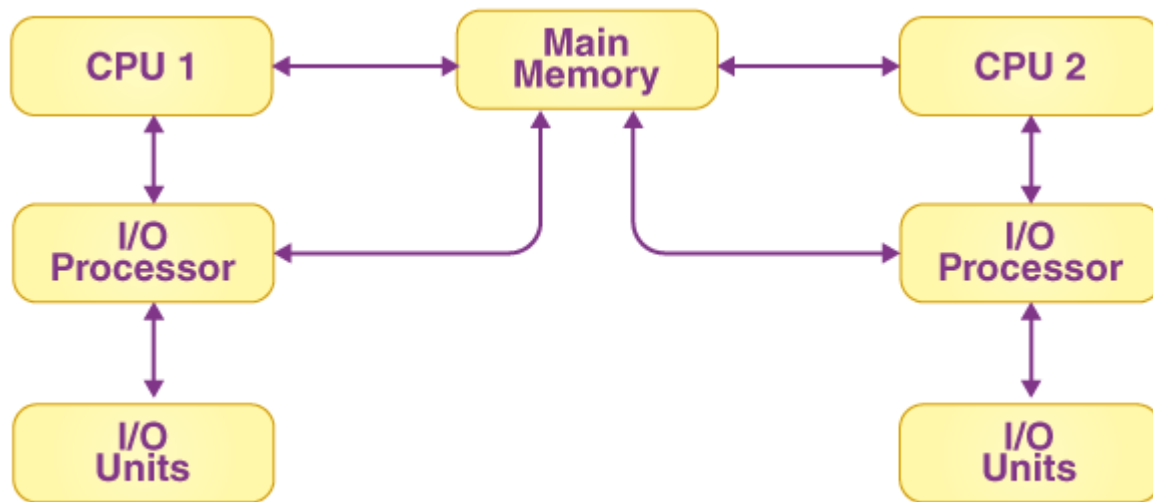
- **Limited Tasks:** Very few tasks run at the same time and their concentration is very less on a few applications to avoid errors.
- **Use heavy system resources:** Sometimes the system resources are not so good and they are expensive as well.
- **Complex Algorithms:** The algorithms are very complex and difficult for the designer to write on.
- **Device driver and interrupt signals:** It needs specific device drivers and interrupts signal to respond earliest to interrupts.
- **Thread Priority:** It is not good to set thread priority as these systems are very less prone to switching tasks.

Multiprocessing Operating System: Multiprocessor Operating Systems are used in operating systems to boost the performance of multiple CPUs within a single computer system. Multiple CPUs are linked together so that a job can be divided and executed more quickly.

Multiprocessor operating systems are used in operating systems to boost the performance of multiple CPUs within a single computer system.

Multiple CPUs are linked together so that a job can be divided and executed more quickly. When a job is completed, the results from all CPUs are compiled to

provide the final output. Jobs were required to share main memory, and they may often share other system resources.

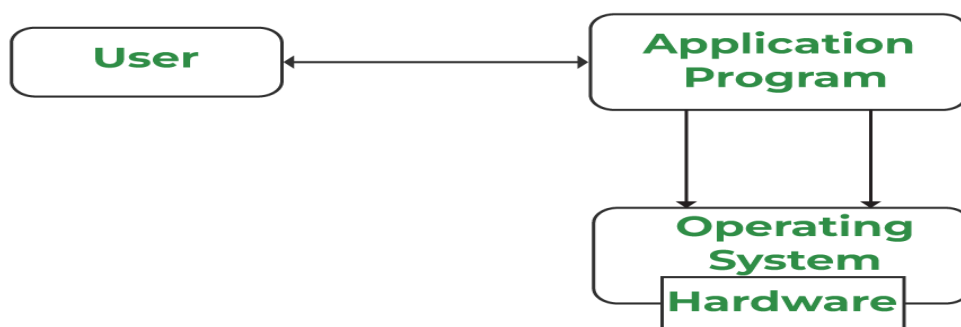


Advantages of Multi-Processing Operating System

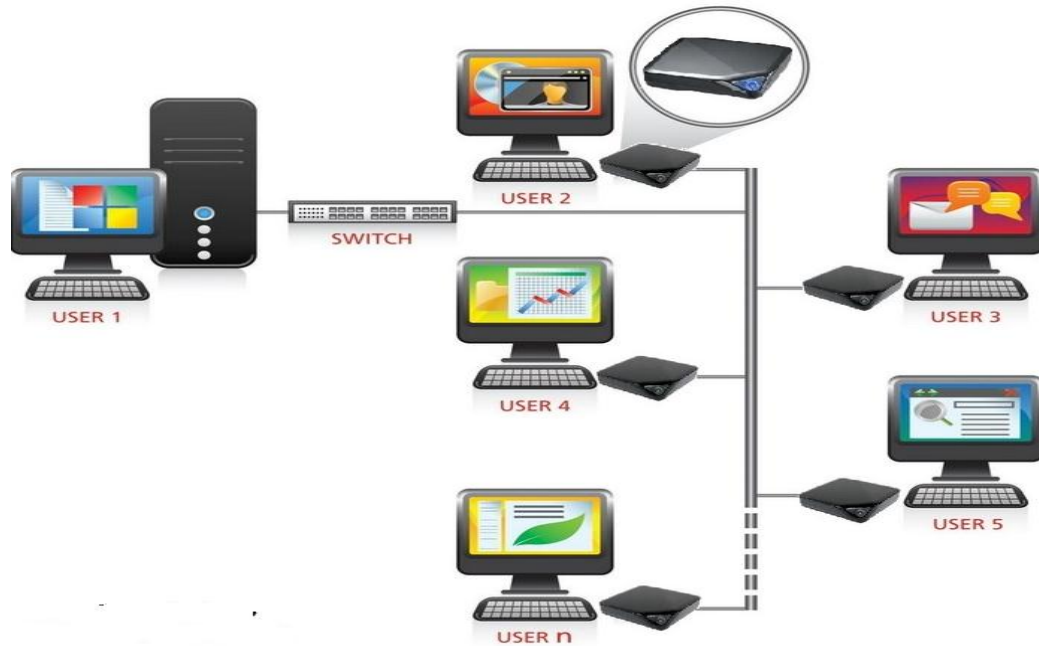
- It increases the throughput of the system as processes can be parallelized.
- As it has several processors, so, if one processor fails, we can proceed with another processor.

Single-User Operating Systems: Single-User Operating Systems are designed to support a single user at a time. Examples include Microsoft Windows for personal computers and Apple macOS.

The operating system for those computers which support only one computer. In this operating system, another user can not interact with another working user. The core part of the single-user operating system is one kernel image that will run at a time i.e there is no other facility to run more than one kernel image.



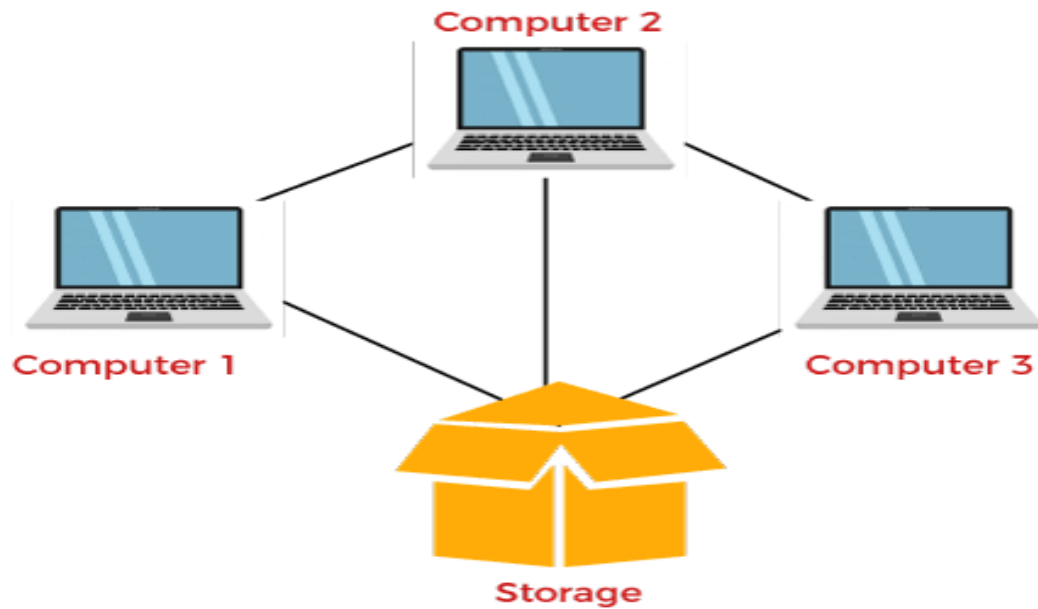
Multi-User Operating Systems: Multi-User Operating Systems are designed to support multiple users simultaneously. Examples include Linux and Unix.



A multi-user operating system is an operating system that allows multiple users to connect and operate a single operating system. The users interact with it through terminals or computers that gave them access to the system through a network or machines such as printers. The operating system should have to meet the requirements of all its users in a balanced way so that if any problem would arise with one user, it does not affect any other user in the chain.

Embedded Operating Systems: An embedded operating system is a computer operating system designed for use in embedded computer systems. These operating systems are designed to be small, resource-efficient, dependable, and reduce many features that aren't required by specialized applications. Examples include Google's Android and Apple's iOS.

Cluster Operating Systems: Cluster Operating Systems are designed to run on a group of computers, or a cluster, to work together as a single system. They are used for high-performance computing and for applications that require high availability and reliability.



Cluster operating systems are a combination of software and hardware clusters. Hardware clusters aid in the sharing of high-performance disks among all computer systems, while software clusters give a better environment for all systems to operate. A cluster system consists of various nodes, each of which contains its cluster software. The cluster software is installed on each node in the clustered system, and it monitors the cluster system and ensures that it is operating properly. If one of the clustered system's nodes fails, the other nodes take over its storage and resources and try to restart.

How to Choose the Operating System.

Price Factor: Price is one of the factors to choose the correct Operating System as there are some OS that is free, like Linux, but there is some more OS that is paid like Windows and macOS.

Accessibility Factor: Some Operating Systems are easy to use like macOS and iOS, but some OS are a little bit complex to understand like Linux. So, you must choose the Operating System in which you are more accessible.

Compatibility factor: Some Operating Systems support very less applications whereas some Operating Systems supports more application. You must choose the OS, which supports the applications which are required by you.

Security Factor: The security Factor is also a factor in choosing the correct OS, as macOS provide some additional security while Windows has little fewer security features.

Examples of Operating Systems

Windows (GUI-based, PC)

GNU/Linux (Personal, Workstations, ISP, File, and print server, Three-tier client/Server)

macOS (Macintosh), used for Apple's personal computers and workstations (MacBook, iMac).

Android(Google's Operating System for smartphones/tablets/smart watches)

iOS (Apple's OS for iPhone, iPad, and iPod Touch).



Module-02**RBT Level-L1, L2****Shell Programming:**

Introduction to Shell Programming using Linux, echo, Variable, User input, Decision Making: if, if else, if elif else, case, Looping: for, while, do while and functions.

If we are using any major operating system, we are indirectly interacting with the shell. While running Ubuntu, Linux Mint, or any other Linux distribution, we are interacting with the shell by using the terminal. In this article we will discuss Linux shells and shell scripting so before understanding shell scripting we have to get familiar with the following terminologies:

- Kernel
- Shell
- Terminal

What is Kernel?

The kernel is a computer program that is the core of a computer's operating system, with complete control over everything in the system. It manages the following resources of the Linux system –

- File management
- Process management
- I/O management
- Memory management
- Device management etc.

Complete Linux system = Kernel + GNU system utilities and libraries + other management scripts + installation scripts.

What is a shell?

Technically speaking in UNIX language, it is an interface between the user and operating system service. A shell provides users with an interface that not only accepts human-readable commands but also executes them.

An operating system consists of many parts, but the 2 core parts are the kernel and shell, we know that the kernel is like the atom of the operating system, the sole heart of the operating system. The kernel works a mediator between the software and hardware.

The shell on the other hand is more like the right hand of the kernel, it takes input from you in the form of commands, processes it, and then gives an output. The shell is retrieved via the terminal that runs it. Shell also provides an interface through which a user works on programs and scripts.

Types of shell

Shell is broadly classified into 2 categories:

- ❖ Command line shell
- ❖ Graphical shell

Command line shell

The shell can be accessed by using a command-line interface, and we all know what a command line is. It is nothing but the terminal in Linux – the lean, mean coding machine. The terminal is a special program to type in the human-readable commands such as **“cat”, “ls”, “cd”, “pwd”, “mkdir”,** and a thousand more!

When you execute the commands, the result is then displayed on the terminal to the user. The terminal is powerful and allows users to store commands in a file and execute them together. Linux is not the only operating system with a command line, even windows had the command prompt and so doe mac OS

The command line is a boon for developers and administrators, but it may not be everyone’s cup of coffee as memorizing so many command sand and an even longer list of options is not an easy task. That is why there are 2 options for the shell so that users can use what they prefer most.

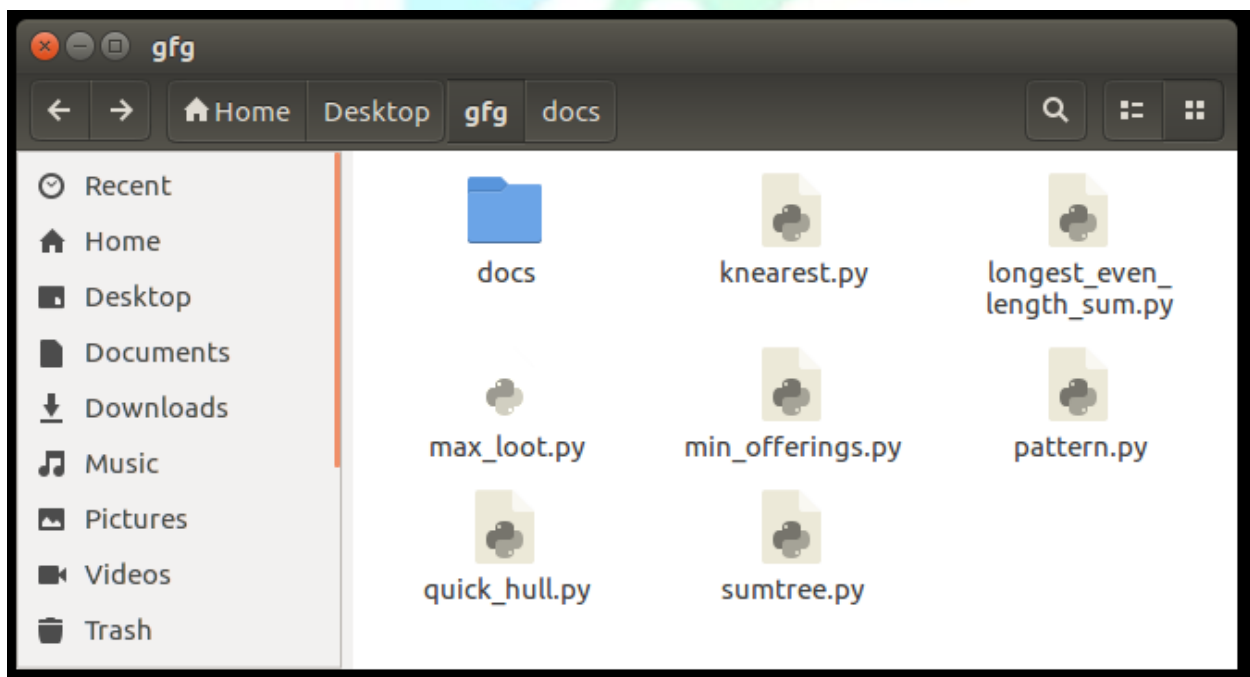
```
override@Atul-HP:~$ ls -l
total 212
drwxrwxr-x  5 override override 4096 May 19 03:45 acadenv
drwxrwxr-x  4 override override 4096 May 27 18:20 acadview_demo
drwxrwxr-x 12 override override 4096 May  3 15:14 anaconda3
drwxr-xr-x  6 override override 4096 May 31 16:49 Desktop
drwxr-xr-x  2 override override 4096 Oct 21 2016 Documents
drwxr-xr-x  7 override override 40960 Jun  1 13:09 Downloads
-rw-r--r--  1 override override 8980 Aug  8 2016 examples.desktop
-rw-rw-r--  1 override override 45005 May 28 01:40 hs_err_pid1971.log
-rw-rw-r--  1 override override 45147 Jun  1 03:24 hs_err_pid2006.log
drwxr-xr-x  2 override override 4096 Mar  2 18:22 Music
drwxrwxr-x 21 override override 4096 Dec 25 00:13 Mydata
drwxrwxr-x  2 override override 4096 Sep 20 2016 newbin
drwxrwxr-x  5 override override 4096 Dec 20 22:44 nltk_data
drwxr-xr-x  4 override override 4096 May 31 20:46 Pictures
drwxr-xr-x  2 override override 4096 Aug  8 2016 Public
drwxrwxr-x  2 override override 4096 May 31 19:49 scripts
drwxr-xr-x  2 override override 4096 Aug  8 2016 Templates
drwxrwxr-x  2 override override 4096 Feb 14 11:22 test
drwxr-xr-x  2 override override 4096 Mar 11 13:27 Videos
drwxrwxr-x  2 override override 4096 Sep  1 2016 xdm-helper
override@Atul-HP:~$
```


In the above screenshot “ls” command with “-l” option is executed. It will list all the files in the current working directory in a long listing format.

Working with a command line shell is a bit difficult for beginners because it's hard to memorize so many commands. It is very powerful; it allows users to store commands in a file and execute them together. This way any repetitive task can be easily automated. These files are usually called batch files in Windows and Shell Scripts in Linux/macOS systems.

Graphical shell

The graphical shell is nothing but the GUI – Graphical interface design. The graphical shell provides an interface for manipulating programs based on the graphical user interface. The GUI allows users to do bath tasks that the terminal can and can not do.



For example, you can do tasks like allowing for operations such as opening, closing, moving, and resizing windows, as well as switching focus between windows and more which the terminal can't. The main advantage of the GUI is that users need not type commands for every small task.

There are various shells that are available for Linux based operating systems, let us look at some of them:

1. BASH (Bourne Again SHell)

The bash shell is the most loved and commonly used shell in Linux-based operating systems. It is a default login shell in Linux systems and in macOS, however, it can also be installed on Microsoft Windows.

The prompt for the bash shell is a dollar sign (\$). The bash shell is a derivative of the Bourne shell, other derivatives of the Bourne shell include POSIX and Korn.

2. KSH (Korn SHell)

The Korn shell is also derived from the Bourne shell but is the base for POSIX. It was written, designed, and developed by David Korn in the early 1980s. Some features of KSH are:

- ❖ It provides object-oriented programming since ksh93t.
- ❖ It has many mathematical and dynamic search functions.
- ❖ It is very similar to C language and even has C-language-like expressions.
- ❖ You get a choice of 3 command line editing styles: vi, Emacs, and Gosling.
- ❖ KSH includes reference variables and hierarchically nested variables.
- ❖ It has associative arrays and built-in floating-point arithmetic operators.
- ❖ The Korn shell provides process substitution and process redirection.

3. CSH (C shell)

The C shell was written, designed, and developed at the University of California by Bill Joy. Just like bash has the prompt "\$", csh has the prompt "%". The C shell was developed to be very similar to the C programming language, it also includes useful programming features like in-built support for arithmetic operations.

The C shell feature 'aliases' and command history which was missing in different types of shells in Linux like the Bourne shell. The complete pathname for the C shell is /bin/sch.

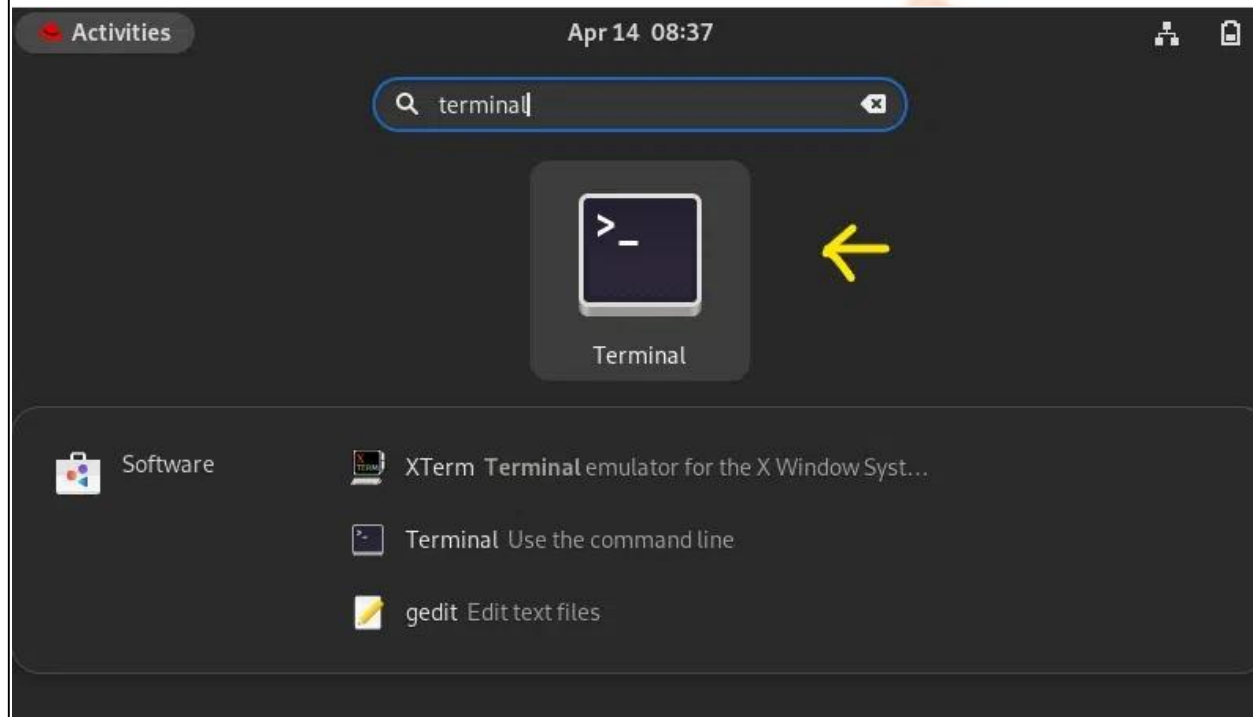
There are many other shells available like:

- ❖ ZSH (Z shell)
- ❖ Fish Shell
- ❖ POSIX
- ❖ TENEX

What is a terminal?

A program which is responsible for providing an interface to a user so that he/she can access the shell. It basically allows users to enter commands and see the output of those commands in a text-based interface. Large scripts that are written to automate and perform complex tasks are executed in the terminal.

To access the terminal, simply search in search box **“terminal”** and double-click it.



Here you can see how the terminal looks of Red Hat Linux.



What is shell scripting?

Shells are interactive, they accept commands as input from users and execute them. But if we want to execute a bunch of commands again and again (like a loop), we have to type in all commands each time in the terminal.

To avoid this, the shell can also take in the commands as the input from a file, where we can compile all of these commands and execute them. These files are called Shell programs or shell scripts and have the extension of **“.sh”**.

Shell programs, just like any other programming language in the world, have a specific syntax and a set of rules. A shell script contains the following elements:

Keywords: if, else, break continue, etc.

Commands: cd, ls, echo, touch, cat, pwd, mkdir, etc.

Loops: if...then...else, while...do, do...while, case, etc.

Functions

Why use shell programming?

There are a gazillion reasons why you should use shell programming, here are a few:

- ❖ Avoids repetitive work and automation
- ❖ Helps in system monitoring
- ❖ Used for routine backups
- ❖ Adding new functionality to shell
- ❖ Creating custom filters
- ❖ Helps in automating various tasks
- ❖ Changing time and date
- ❖ Printing current working directory
- ❖ Scheduling jobs

Advantages of shell programming

- ❖ Easy to use
- ❖ Portable
- ❖ Quick start
- ❖ Interactive debugging
- ❖ Same syntax as that of command line
- ❖ Avoids repetitive work and automation
- ❖ Shell scripting is much quicker than writing commands in the terminal
- ❖ Simple up learn

Disadvantages of shell programming

- ❖ They say that every good thing has its negatives, which also hold true for shell programming, here are some of its disadvantages:
- ❖ Slow execution speed
- ❖ Not suited for large and complex tasks
- ❖ Design flaws within the language syntax
- ❖ Provides minimal data structure, unlike other scripting languages
- ❖ Prone to costly errors

Creating a shell program

To write a shell program we need to write it in a **“.sh”** file, meaning that a regular **“.txt” or “.odt” or “.doc”** will not support. Now there are a hundred ways to create a **“.sh”** file, let us look at some alternatives:

Using ‘gedit’

- ✓ One method of creating a **“.sh”** file is by using the edit command.
- ✓ You can directly create and open a **“.sh”** file by specifying the name of the file as shown below, or you can simply use the edit command and save the file and name it later.

```
gopikiran@dataflair:~$ gedit program.sh
```

You must be careful while naming your file because if you write the name of an already existing file with the command, it will open the existing file instead of creating one.

Using ‘nano editor’

- ✓ Another method to create a **“.sh”** file is by using the **nano editor** in Linux-based operating systems.
- ✓ It works similar to the gedit command where you specify the name of the file you want to create next to the command.

Similar to the gedit command, even the nano command will open the existing file instead of creating a new one if you enter the name of an already existing file.

```
gopikiran@dataflair:~$ nano program.sh
gopikiran@dataflair:~$
```


Using Vi editor

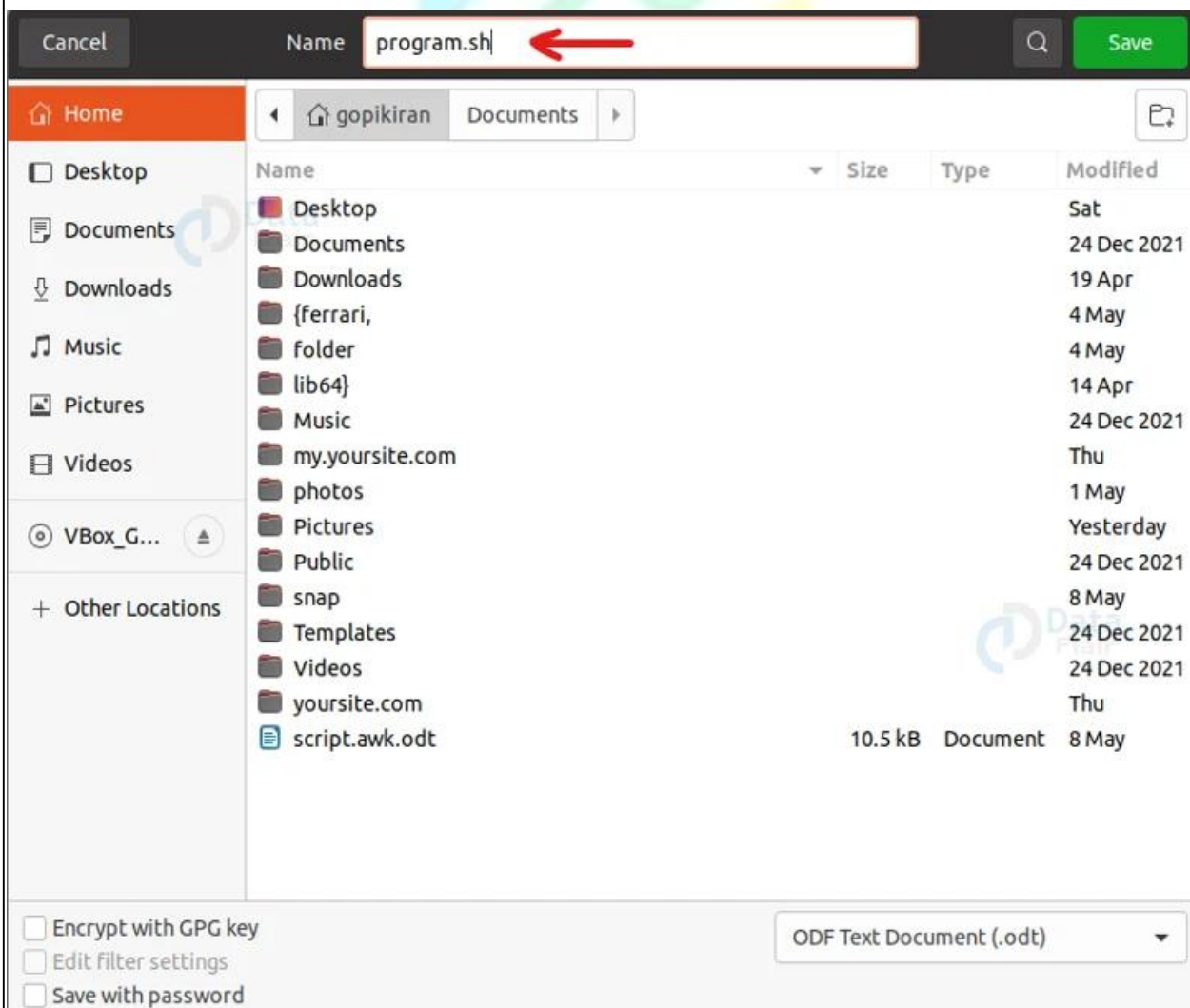
- ✓ We all know that the **Vi** editor is a really powerful editor for **text manipulation**, but not that easy for **text creation**, meaning it will be even harder to write shell programs in it.
- ✓ The **'vi'** command, **specify the name of the file.**

Even the **Vi editor** will open the existing file if you enter the name of an already existing file.

```
gopikiran@dataflair:~$ vi program.sh
```

Using the GUI

All the distributions of Linux come with Libre Office preinstalled, you can simply open a new file, write your program, and while saving your file, simply add the extension “.sh” to your filename as shown:



echo command in Linux with Examples

The **echo** command in Linux is a built-in command that **allows users to display lines of text or strings** that are passed as arguments.

Syntax of `echo` command in Linux

echo [option] [string]

Here,

[options] = The various options available for modifying the behavior of the `echo` command

[string] = It is the string that we want to display.

Displaying Text/String

If we want to display **"DYPIU"**. We use the following command.

```
echo "DYPIU"
```

Options Available in `echo` command in Linux:

- ✓ **\b** : it removes all the spaces in between the text.

Example: **echo -e "Geeks \bfor \bGeeks"**

```
raghvendra@raghvendra-Inspiron-15-3567:~$ echo -e "Geeks \bfor \bGeeks"
GeeksforGeeks
raghvendra@raghvendra-Inspiron-15-3567:~$
```

- ✓ **\c** : suppress trailing new line with backspace interpreter '-e' to continue without emitting new line.

Example: **echo -e "Geeks \cfor Geeks"**

```
raghvendra@raghvendra-Inspiron-15-3567:~$ echo -e "Geeks \cfor Geeks"
Geeks raghvendra@raghvendra-Inspiron-15-3567:~$
```

- ✓ **\n** : this option creates a new line from where it is used.

Example: **echo -e "Geeks \nfor \nGeeks"**

```
raghvendra@raghvendra-Inspiron-15-3567:~$ echo -e "Geeks \nfor \nGeeks"
Geeks
for
Geeks
raghvendra@raghvendra-Inspiron-15-3567:~$
```

- ✓ **\t** : this option is used to create horizontal tab spaces.

Example: `echo -e "Geeks \tfor \tGeeks"`

```
raghvendra@raghvendra-Inspiron-15-3567:~$ echo -n "Geeks for Geeks"
Geeks for Geeksraghvendra@raghvendra-Inspiron-15-3567:~$
```

- ✓ **\r** : carriage return with backspace interpreter ‘-e’ to have specified carriage return in output.

Example: `echo -e "Geeks \rfor Geeks"`

In the above example, text before \r is not printed.

```
raghvendra@raghvendra-Inspiron-15-3567:~$ echo -e "Geeks \rfor Geeks"
for Geeks
raghvendra@raghvendra-Inspiron-15-3567:~$
```

- ✓ **\v** : this option is used to create vertical tab spaces.

Example: `echo -e "Geeks \vfor \vGeeks"`

```
raghvendra@raghvendra-Inspiron-15-3567:~$ echo -e "Geeks \vfor \vGeeks"
Geeks
    for
        Geeks
raghvendra@raghvendra-Inspiron-15-3567:~$
```

- ✓ **\a** : alert return with backspace interpreter ‘-e’ to have sound alert.

Example: `echo -e "\aGeeks for Geeks"`

This command when executed, it will produce an alert sound or Bell.

```
raghvendra@raghvendra-Inspiron-15-3567:~$ echo -e "\aGeeks for Geeks"
Geeks for Geeks
raghvendra@raghvendra-Inspiron-15-3567:~$
```

- ✓ **echo ***: this command will print all files/folders, similar to ls command.

Example: `echo *`

```
raghvendra@raghvendra-Inspiron-15-3567:~$ echo *
Desktop Documents Downloads git_hand git_repos java2python-0.5.1 Music My songs
Pictures Public snap Templates Videos
raghvendra@raghvendra-Inspiron-15-3567:~$ ls
Desktop    git_hand      Music        Public       Videos
Documents  git_repos     'My songs'   snap
Downloads  java2python-0.5.1 Pictures     Templates
raghvendra@raghvendra-Inspiron-15-3567:~$
```

- ✓ **-n**: this option is used to omit echoing trailing newline.

Variables

A shell variable is a **character string** in a shell that **stores some value**. It could be an **integer, filename, string, or some shell command itself**.

Rules for variable definition.

- ✓ A variable name could contain any **alphabet (a-z, A-Z)**, any **digits (0-9)**, and an underscore (**_**).
- ✓ However, a variable name must start with an **alphabet or underscore**. It can **never start with a number**.

Some examples of valid and invalid variable names:

Valid Variable Names

- 1) ABC
- 2) _AV_3
- 3) AV232

Invalid variable names

- 1) 2_AN
- 2) !ABD
- 3) \$ABC, &QAID

Variable Types

Three main types of variables:

- ❖ Local Variable
- ❖ Environment Variable
- ❖ Shell Variables

Local Variable:

Variables which are specific to the current instance of shell. They are basically used within the shell, but not available for the program or other shells that are started from within the current shell.

For example:

```
`name=DYPIU`
```

In this case the local variable is **(name)** with the value of **DYPIU**. Local variables is temporary storage of data within a shell script.

Environment Variable:

These variables are commonly used to **configure the behavior script and programs** that are run by shell. Environment variables are only created once, after which they can be used by any user.

For example:

```
`export PATH=/usr/local/bin:$PATH` would add `/usr/local/bin` to the beginning of the shell's search path for executable programs.
```

Shell Variables:

Variables that are set by shell itself and help shell to work with functions correctly. It contains both, which means it has both, some variables are Environment variable, and some are Local Variables.

For example:

`\$PWD` = Stores working directory

`\$HOME` = Stores user's home directory

`\$SHELL` = Stores the path to the shell program that is being used.

What is Shell Variable Used For?

Shell Variables are used to store data and information within a shell (terminal), and they are also used for controlling the behavior of program and scripts.

Some of the common uses are:

- Setting environment variables.
- Storing configuration data.
- Storing temporary data.
- Passing arguments to scripts.

What is Shell Variable and Shell Scripting?

Shell Variable is used in shell scripts for many functionalities like storing data and information, taking input from users, printing values that are stored. They are also used for storing data temporarily and storing output of commands.

Shell Scripting is a way of writing scripts of programs that are executed in a terminal or shell. Basically, it is a program or script which is written with the help of variables mentioned in it. It is powerful because it can automate tasks, and in this one can use programming constructs that are available in shell, such as loops, conditionals and functions.

Read User Input

If we want to request input from the user, we will use the read command. This command accepts input and saves it to a variable. It reads a single line from the Bash shell.

Now let's understand the basic keywords of bash commands so that we can learn Shell Scripting - Read User Input:

Commands	Description
Echo	It displays the argument on standard output.
pwd(print working directory)	It displays the current working directory.
ls(list)	It lists the content of the directory.
cd(change directory)	It is used to change the directory.
mkdir(make directory)	It is used to make a directory.
rmdir(remove directory)	It is used to remove the directory.
touch	It is used to create blank files.
rm(remove)	It removes the files from a directory.
mv(move)	It is used to move the file from one directory to another.
p	It is used to prevent shell reading from user-controlled files.
\$	It is used to print the variable values.
sp	It is used to keep the user information hidden.

Which one is better in Shell Scripting - Read User Input?

Command-line arguments should be the first priority as they are the most convenient for users. This is because the data is saved in their command history, allowing them to return to data quickly to the user.

But in this case, if you want to take the inputs like passwords, then reading inputs during script execution will be the better option as it hides the data from being displayed.

Decision Making: if , if else, if elif else, case

In programming, Decision making is one of the important concepts. The programmer provides one or more conditions for the execution of a block of code. If the conditions are satisfied then those block of codes only gets executed. Two types of decision-making statements are used within shell scripting. They are –

- If-else statement
- case-sac statement

If-else statement

If else statement is a conditional statement. It can be used to execute two different codes based on whether the given condition is satisfied or not. There are a couple of varieties present within the if-else statement. They are –

- if
- if-else-if
- if-elif-else-if
- nested if-else

if : This block will process if specified condition is true.

Syntax:

```
if [ expression ]  
then  
    statement  
if
```

Example 1: Implementing if statement

```
#Initializing two variables  
a=10  
b=20  
  
#Check whether they are equal  
if [ $a == $b ]  
then  
    echo "a is equal to b"  
if  
  
#Check whether they are not equal  
if [ $a != $b ]  
then  
    echo "a is not equal to b"  
if
```

Output:

```
$bash -f main.sh  
a is not equal to b
```

if-else statement: If specified condition is not true in if part then else part will be execute

Syntax:

```
if [ expression ]
then
    statement1
else
    statement2
if
```

Example 2: Implementing if.else statement

#initializing two variables

```
a=20
b=20
if [ $a == $b ]
then
```

#If they are equal then print this

```
echo "a is equal to b"
else
#else print this
echo "a is not equal to b"
```

if

Output:

```
$bash -f main.sh
a is equal to b
```

if..elif..else..if statement (Else If ladder)

To use multiple conditions in one if-else block, then elif keyword is used in shell. If expression1 is true then it executes statement 1 and 2, and this process continues. If none of the condition is true then it processes else part.

Syntax:

```
if [ expression1 ]
then
    statement1
    statement2
    .
    .
elif [ expression2 ]
then
    statement3
```

```
        statement4
        .
        .
    else
        statement5
    if
```

Example 4: Age=17

```
if [ "$Age" -ge 18 ]; then
    echo "You can vote"
elif [ "$Age" -eq 17 ]; then
    echo "You can vote after one year" else echo "You cannot vote"
if
```

Output:

You can Vote after one year

if..then..else..if..then..if..if..(Nested if)

Nested if-else block can be used when, one condition is satisfies then it again checks another condition. In the syntax, if expression1 is false then it processes else part, and again expression2 will be check.

Syntax:

```
if [ expression1 ]
then
    statement1
    statement2
    .
else
    if [ expression2 ]
    then
        statement3
        .
    if
```

Example 4:

```
echo "Enter subject"
read subject
if [ $subject == 'Linux' ]
then
    echo "Enter Marks"
    read marks if [ $marks -ge 30 ]
```

```
then
echo "You passed"
else
echo "You failed"
if
else
echo "Wrong Subject"
if
```

Output:

OUTPUT1	OUTPUT2	OUTPUT3
Enter subject	Enter subject	Enter subject
Linux	Linux	DBMS
Enter Marks	Enter Marks	Wrong subject
97	29	
You passed	You failed	

switch statement:

case statement works as a switch statement if specified value match with the pattern then it will execute a block of that particular pattern

When a match is found all of the associated statements until the double semicolon (;;) is executed.

A case will be terminated when the last command is executed.

If there is no match, the exit status of the case is zero.

Syntax:

```
case in
Pattern 1) Statement 1;;
Pattern n) Statement n;;
Esac
```

Example 5: Implementing switch statement

```
CARS="bmw"
```

```
#Pass the variable in string
```

```
case "$CARS" in
```

```
#case 1
```

```
"mercedes") echo "Headquarters - Affalterbach, Germany" ;;
```

```
#case 2
```

```
"audi") echo "Headquarters - Ingolstadt, Germany" ;;
```



```
#case 3
"bmw") echo "Headquarters - Chennai, Tamil Nadu, India" ;;
esac
```

Output:

```
$bash -f main.sh
Headquarters - Chennai, Tamil Nadu, India.
```

Looping: for, while, do while and functions.**‘for’ statement in Shell Script in Linux**

The for loop operates on lists of items. It repeats a set of commands for every item in a list.

Here var is the name of a variable and word1 to wordN are sequences of characters separated by spaces (words). Each time the for loop executes, the value of the variable var is set to the next word in the list of words, word1 to wordN.

Syntax:

```
#!/bin/bash
for <var> in <value1 value2 ... valuen>
do
    <command 1>
    <command 2>
    <etc>
done
```

Implementation of ‘for’ Loop with ‘break’ statement in Shell Script.

First, we create a file using a text editor in Linux. In this case, we are using ‘vim’ editor.

vim for.sh

you can replace “for.sh” with the desired name.

Then we make our script executable using the ‘chmod’ command in Linux.

chmod +x for.sh

```
#!/bin/bash
#Start of for loop
for a in 1 2 3 4 5 6 7 8 9 10
```

```
do
    # if a is equal to 5 break the loop
    if [ $a == 5 ]
    then
        break
    fi
# Print the value
    echo "Iteration no $a"
done
```

Output:

```
administrator@GFG19566-LAPTOP:~$ vim for.sh
administrator@GFG19566-LAPTOP:~$ chmod +x for.sh
administrator@GFG19566-LAPTOP:~$ ./for.sh
Iteration no 1
Iteration no 2
Iteration no 3
Iteration no 4
administrator@GFG19566-LAPTOP:~$
```

Implementation of `for` Loop with `continue` statement in Shell Script.

```
#!/bin/bash
for a in 1 2 3 4 5 6 7 8 9 10
do
    # if a = 5 then continue the loop and
    # don't move to line 8
    if [ $a == 5 ]
    then
        continue
    fi
    echo "Iteration no $a"
done
```

Output:

```
administrator@GFG19566-LAPTOP:~$ vim for_continue.sh
administrator@GFG19566-LAPTOP:~$ chmod +x for_continue.sh
administrator@GFG19566-LAPTOP:~$ ./for_continue.sh
Iteration no 1
Iteration no 2
Iteration no 3
Iteration no 4
Iteration no 6
Iteration no 7
Iteration no 8
Iteration no 9
Iteration no 10
```

while loop

The **while loop** enables you to execute a set of commands repeatedly until some condition occurs. It is usually used when you need to manipulate the value of a variable repeatedly.

Syntax

```
while command
do
    Statement(s) to be executed if command is true
done
```

Here the Shell command is evaluated. If the resulting value is true, given statement(s) are executed. If command is false then no statement will be executed and the program will jump to the next line after the done statement.

Example: To display the numbers zero to nine

```
#!/bin/sh
a=0
while [ $a -lt 10 ]
do
    echo $a
    a=`expr $a + 1`
done
```

Output:

0
1
2
3
4
5
6
7
8
9

Each time this loop executes, the variable **a** is checked to see whether it has a value that is less than 10. If the value of **a** is less than this test condition has an exit status of **0**. In this case, the current value of **a** is displayed and later a is incremented by **1**.

Functions

A function is a collection of statements that execute a specified task. Its main goal is to break down a complicated procedure into simpler subroutines that can subsequently be used to accomplish the more complex routine.

Basic Structure of a Function in Shell Scripting

The basic syntax for defining a function in shell scripting is:

```
function_name()
{
    // body of the function
}
```

The **function_name** can be any valid string and the body can be any sequence of valid statements in the scripting language.

The **body of the function** can include any sequence of valid shell commands or statements.

Pass Parameters to a Function

We can define a function that will accept parameters while calling the function. These parameters would be represented by **\$1**, **\$2** and so on.

Example:#!/bin/sh

Define your function here

```
Hello () {
    echo "Hello World $1 $2"
}
```

Upon execution, you will receive the following result –**Hello World**

Functions with Passing Arguments

Syntax of Functions with Passing Arguments:

```
#for defining
function_name(){
    ....
    parameter_1 = $1
    parameter_2 = $2
    .
    .
    .
    parameter_n = $n
    ....
    commands
    ....
}
```

Example:#!/bin/bash

```
add_two_num(){
    local sum=$(( $1+$2 ))
    echo sum of $1 and $2 is $sum
}
```

add_two_num 2 3

Output:sum of 2 and 3 is 5

```
satyajit@satyajit:~/Desktop/Bash$ ./gfg.sh
sum of 2 and 3 is 5
satyajit@satyajit:~/Desktop/Bash$
```

```
1 #!/bin/bash
2
3 add_two_num(){
4     local sum=$(( $1+$2 ))
5     echo sum of $1 and $2 is $sum
6 }
7
8 add_two_num 2 3
```


Functions with Return Values

A return value is produced and returned to the calling method by a function after it completes its execution. A return value can be used to share the produced result or some status code about whether one function was successfully executed or not. The return value is assigned to the '\$?' variable.

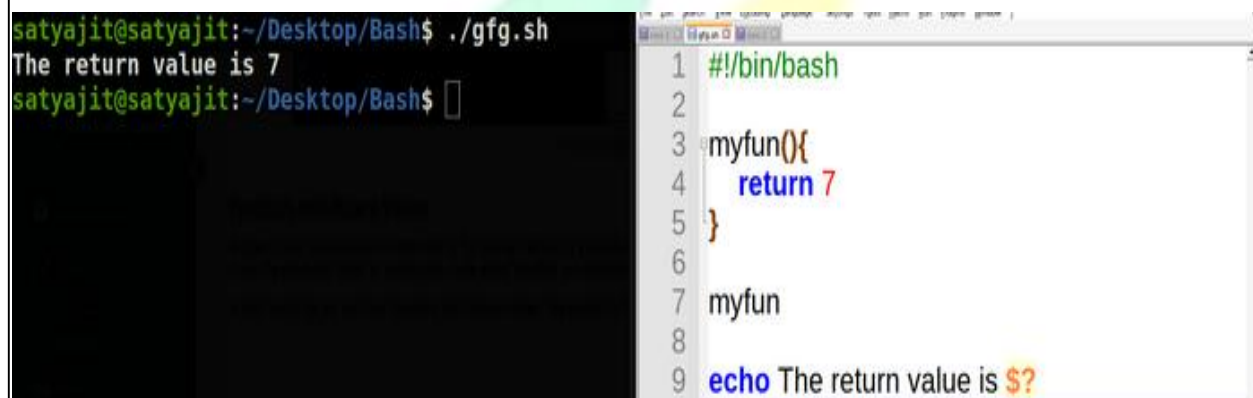
Example: #!/bin/bash

```
myfun(){  
    return 7  
}
```

myfun

echo The return value is \$?

Output: The return value is 7



```
satyajit@satyajit:~/Desktop/Bash$ ./gfg.sh  
The return value is 7  
satyajit@satyajit:~/Desktop/Bash$
```

```
1 #!/bin/bash  
2  
3 myfun(){  
4     return 7  
5 }  
6  
7 myfun  
8  
9 echo The return value is $?
```

Module-03**RBT Level-L1, L2****Operating System Services:**

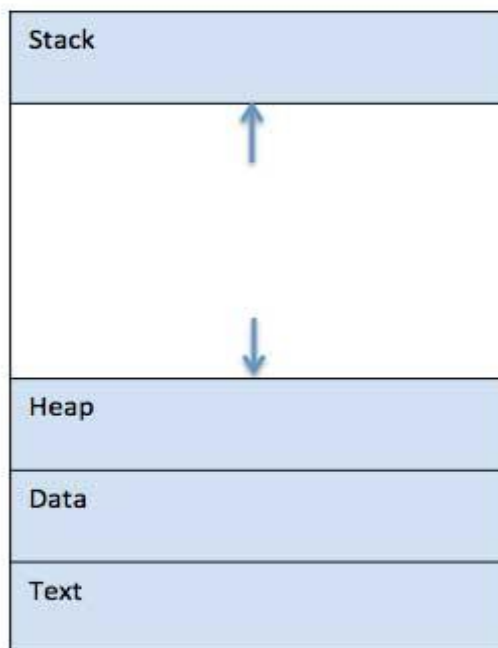
Processes, Process structure, Process Scheduling, Scheduling algorithms, Process Synchronization & Deadlocks, Thread Management, Memory Management, System Calls, File System.

Process in an Operating System

A process is essentially running software. **OR** A process refers to an entity that helps in representing the fundamental unit of work that must be implemented in any system.

Example: Consider we write our computer programs in a text file and when we execute this program, it becomes a process which performs all the tasks mentioned in the program.

When a program is loaded into the memory and it becomes a process, it can be divided into four sections – stack, heap, text and data. The following image shows a simplified layout of a process inside main memory –

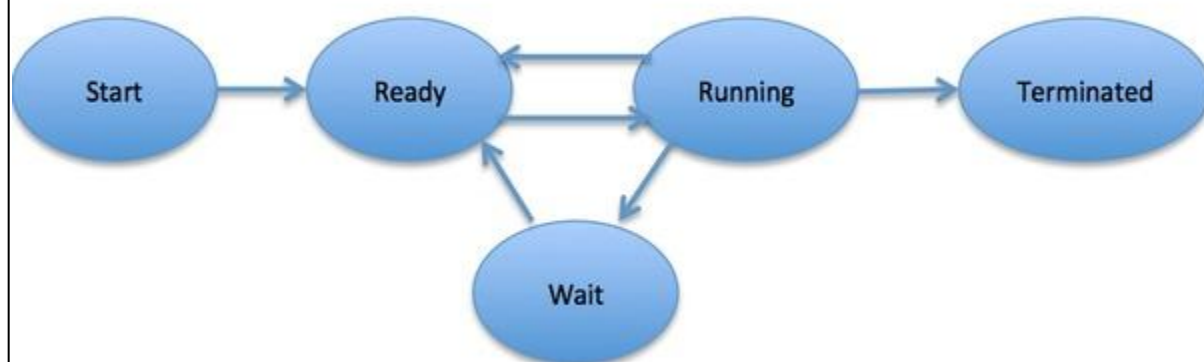


S.N.	Component & Description
1	Stack The process Stack contains the temporary data such as method/function parameters, return address and local variables.
2	Heap This is dynamically allocated memory to a process during its run time.
3	Text This includes the current activity represented by the value of Program Counter and the contents of the processor's registers.
4	Data This section contains the global and static variables.

Process structure or Process Life Cycle.

When a process executes, it passes through different states. These stages may differ in different operating systems, and the names of these states are also not standardized.

In general, a process can have one of the following five states at a time.



S.N.	State & Description
1	Start This is the initial state when a process is first started/created.
2	Ready The process is waiting to be assigned to a processor. Ready processes are waiting to have the processor allocated to them by the operating system so that they can run. Process may come into this state after Start state or while running it by but interrupted by the scheduler to assign CPU to some other process.
3	Running Once the process has been assigned to a processor by the OS scheduler, the process state is set to running and the processor executes its instructions.
4	Waiting Process moves into the waiting state if it needs to wait for a resource, such as waiting for user input, or waiting for a file to become available.
5	Terminated or Exit Once the process finishes its execution, or it is terminated by the operating system, it is moved to the terminated state where it waits to be removed from main memory.

Process Scheduling.

Definition

The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process on the basis of a particular strategy.

Process scheduling is an essential part of a Multiprogramming operating systems. Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using time multiplexing.

Categories of Scheduling

There are two categories of scheduling:

- 1) **Non-preemptive**
- 2) **Preemptive**

Non-preemptive: Here the resource can't be taken from a process until the process completes execution. The switching of resources occurs when the running process terminates and moves to a waiting state.

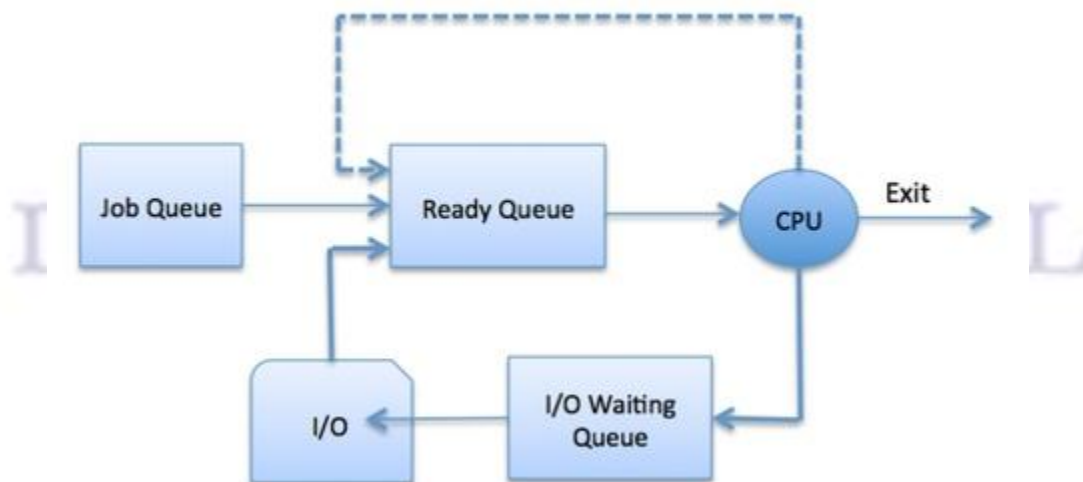
Preemptive: Here the OS allocates the resources to a process for a fixed amount of time. During resource allocation, the process switches from running state to ready state or from waiting state to ready state. This switching occurs as the CPU may give priority to other processes and replace the process with higher priority with the running process.

Difference Between Preemptive and Non-Preemptive Scheduling

Preemptive Scheduling	Non-Preemptive Scheduling
The CPU is allocated to the processes for a certain amount of time.	The CPU is allocated to the process till it ends its the fewer execution or switches to waiting state.
The executing process here is interrupted in the middle of execution.	The executing process here is not interrupted in the middle of execution.
It usually switches the process from the ready state to the running state, vice-versa, and maintains the ready queue.	It does not switch the process from running state to a ready state.
It is quite flexible because the critical processes are allowed to access CPU as they arrive	It is rigid as even if a critical process enters the ready queue the process running

into the ready queue, no matter what process is executing currently.	CPU is not disturbed.
Preemptive scheduling allows a process to be interrupted in the midst of its execution, taking the CPU away and allocating it to another process.	Non-preemptive scheduling ensures that a process relinquishes control of the CPU only when it finishes with its current CPU burst.
Preemptive scheduling incurs a cost associated with accessing shared data.	Non-preemptive scheduling does not increase the cost.
It also affects the design of the operating system Kernel.	It does not affect the design of the OS kernel.
Example: Round robin method.	Example: First come first serve method.

Process Scheduling Queues



The Operating System maintains the following important process scheduling queues –

Job queue – This queue keeps all the processes in the system.

Ready queue – This queue keeps a set of all processes residing in main memory, ready and waiting to execute. A new process is always put in this queue.

Device queues – The processes which are blocked due to unavailability of an I/O device constitute this queue.

Schedulers

Schedulers are special system software which handle process scheduling in various ways. Their main task is to select the jobs to be submitted into the system and to decide which process to run. Schedulers are of three types –

- Long-Term Scheduler
- Short-Term Scheduler
- Medium-Term Scheduler

Long Term Scheduler: It is also called a job scheduler. A long-term scheduler determines which programs are admitted to the system for processing. It selects processes from the queue and loads them into memory for execution. Process loads into the memory for CPU scheduling.

Short Term Scheduler: It is also called as CPU scheduler. Its main objective is to increase system performance in accordance with the chosen set of criteria. It is the change of ready state to running state of the process. CPU scheduler selects a process among the processes that are ready to execute and allocates CPU to one of them.

Medium Term Scheduler: Medium-term scheduling is a part of swapping. It removes the processes from the memory. It reduces the degree of multiprogramming. The medium-term scheduler is in-charge of handling the swapped out-processes.

Comparison among Scheduler:

S.N	Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
01	It is a job scheduler	It is a CPU scheduler	It is a process swapping scheduler.
02	Speed is lesser than short term scheduler	Speed is fastest among other two	Speed is in between both short and long term scheduler.
03	It controls the degree of multiprogramming	It provides lesser control over degree of multiprogramming	It reduces the degree of multiprogramming.
04	It is almost absent or minimal in time sharing system	It is also minimal in time sharing system	It is a part of Time sharing systems.
05	It selects processes from pool and loads them into memory for execution	It selects those processes which are ready to execute	It can re-introduce the process into memory and execution can be continued.

Scheduling Algorithms:

- **First Come First Serve(FCFS)**
- **Shortest Job First(SJF)**
- **Longest Job First(LJF)**
- **Priority Scheduling**
- **Round robin**
- **Shortest Remaining Time First**
- **Longest Remaining Time First**
- **Highest Response Ratio Next**
- **Multiple Queue Scheduling**

First Come First Serve(FCFS)

- Jobs are executed on first come, first serve basis.
- It is a non-preemptive, pre-emptive scheduling algorithm.
- Easy to understand and implement.
- Its implementation is based on FIFO queue.
- Poor in performance as average wait time is high.

Characteristics of FCFS:

- FCFS supports non-preemptive and preemptive CPU scheduling algorithms.
- Tasks are always executed on a First-come, First-serve concept.
- FCFS is easy to implement and use.
- This algorithm is not much efficient in performance, and the wait time is quite high.

Advantages of FCFS:

- Easy to implement
- First come, first serve method

Disadvantages of FCFS:

- FCFS suffers from Convoy effect.
- The average waiting time is much higher than the other algorithms.
- FCFS is very simple and easy to implement and hence not much efficient.

Shortest Job First(SJF)

- This is a non-preemptive, pre-emptive scheduling algorithm.
- Best approach to minimize waiting time.
- Easy to implement in Batch systems where required CPU time is known in advance.
- Impossible to implement in interactive systems where required CPU time is not known.
- The processor should know in advance how much time process will take.

Characteristics of SJF:

- Shortest Job first has the advantage of having a minimum average waiting time among all operating system scheduling algorithms.
- It is associated with each task as a unit of time to complete.
- It may cause starvation if shorter processes keep coming. This problem can be solved using the concept of ageing.

Advantages of Shortest Job first:

- As SJF reduces the average waiting time thus, it is better than the first come first serve scheduling algorithm.
- SJF is generally used for long term scheduling

Disadvantages of SJF:

- One of the demerit SJF has is starvation.
- Many times it becomes complicated to predict the length of the upcoming CPU request

Longest Job First(LJF)

- This is a non-preemptive, pre-emptive scheduling algorithm.
- Best approach to minimize waiting time.
- Easy to implement in Batch systems where required CPU time is known in advance.
- Impossible to implement in interactive systems where required CPU time is not known.
- The processor should know in advance how much time process will take.

Characteristics of LJF:

- Among all the processes waiting in a waiting queue, CPU is always assigned to the process having largest burst time.
- If two processes have the same burst time then the tie is broken using FCFS i.e. the process that arrived first is processed first.
- LJF CPU Scheduling can be of both preemptive and non-preemptive types.

Advantages of LJF:

- No other task can schedule until the longest job or process executes completely.
- All the jobs or processes finish at the same time approximately.

Disadvantages of LJF:

- Generally, the LJF algorithm gives a very high average waiting time and average turn-around time for a given set of processes.
- This may lead to convoy effect.

Priority Scheduling

- Priority scheduling is a non-preemptive algorithm and one of the most common scheduling algorithms in batch systems.
- Each process is assigned a priority. Process with highest priority is to be executed first and so on.
- Processes with same priority are executed on first come first served basis.
- Priority can be decided based on memory requirements, time requirements or any other resource requirement.

Characteristics of Priority Scheduling:

- Schedules tasks based on priority.
- When the higher priority work arrives and a task with less priority is executing, the higher priority process will take the place of the less priority process and
- The later is suspended until the execution is complete.
- Lower is the number assigned, higher is the priority level of a process.

Advantages of Priority Scheduling:

- The average waiting time is less than FCFS
- Less complex

Disadvantages of Priority Scheduling:

- One of the most common demerits of the Preemptive priority CPU scheduling algorithm is the Starvation Problem. This is the problem in which a process has to wait for a longer

amount of time to get scheduled into the CPU. This condition is called the starvation problem.

Round robin

- Round Robin is the preemptive process scheduling algorithm.
- Each process is provided a fix time to execute, it is called a quantum.
- Once a process is executed for a given time period, it is preempted and other process executes for a given time period.
- Context switching is used to save states of preempted processes.

Characteristics of Round robin:

- It's simple, easy to use, and starvation-free as all processes get the balanced CPU allocation.
- One of the most widely used methods in CPU scheduling as a core.
- It is considered preemptive as the processes are given to the CPU for a very limited time.

Advantages of Round robin:

- Round robin seems to be fair as every process gets an equal share of CPU.
- The newly created process is added to the end of the ready queue.

Shortest Remaining Time First

It is the preemptive form of SJF. In this algorithm, the OS schedules the Job according to the remaining time of the execution.

Characteristics of Shortest remaining time first:

- SRTF algorithm makes the processing of the jobs faster than SJF algorithm, given it's overhead charges are not counted.
- The context switch is done a lot more times in SRTF than in SJF and consumes the CPU's valuable time for processing. This adds up to its processing time and diminishes its advantage of fast processing.

Advantages of SRTF:

- In SRTF the short processes are handled very fast.
- The system also requires very little overhead since it only makes a decision when a process completes or a new process is added.

Disadvantages of SRTF:

- Like the shortest job first, it also has the potential for process starvation.
- Long processes may be held off indefinitely if short processes are continually added.

Longest Remaining Time First

The longest remaining time first is a preemptive version of the longest job first scheduling algorithm. This scheduling algorithm is used by the operating system to program incoming processes for use in a systematic way. This algorithm schedules those processes first which have the longest processing time remaining for completion.

Characteristics of longest remaining time first:

- Among all the processes waiting in a waiting queue, the CPU is always assigned to the process having the largest burst time.
- If two processes have the same burst time then the tie is broken using FCFS i.e. the process that arrived first is processed first.
- LRTF CPU Scheduling can be of both preemptive and non-preemptive.
- No other process can execute until the longest task executes completely.
- All the jobs or processes finish at the same time approximately.

Disadvantages of LRTF:

- This algorithm gives a very high average waiting time and average turn-around time for a given set of processes.
- This may lead to a convoy effect.

Highest Response Ratio Next

In this scheduling Algorithm, the process with highest response ratio will be scheduled next. This reduces the starvation in the system.

Characteristics of Highest Response Ratio Next:

- The criteria for HRRN is Response Ratio, and the mode is Non-Preemptive.
- HRRN is considered as the modification of Shortest Job First to reduce the problem of starvation.
- In comparison with SJF, during the HRRN scheduling algorithm, the CPU is allotted to the next process which has the highest response ratio and not to the process having less burst time.

Advantages of HRRN:

- HRRN Scheduling algorithm generally gives better performance than the shortest job first Scheduling.
- There is a reduction in waiting time for longer jobs and also it encourages shorter jobs.

Disadvantages of HRRN:

- The implementation of HRRN scheduling is not possible as it is not possible to know the burst time of every job in advance.
- In this scheduling, there may occur an overload on the CPU.

Multiple Queue Scheduling

- Multiple queues are maintained for processes with common characteristics.
- Each queue can have its own scheduling algorithms.
- Priorities are assigned to each queue.

Advantages of multilevel queue scheduling:

- The main merit of the multilevel queue is that it has a low scheduling overhead.

Disadvantages of multilevel queue scheduling:

- Starvation problem
- It is inflexible in nature

Process Synchronization & Deadlocks

Process Synchronization is the coordination of execution of multiple processes in a multi-process system to ensure that they access shared resources in a controlled and predictable manner.

The main objective of process synchronization is to ensure that multiple processes access shared resources without interfering with each other and to prevent the possibility of inconsistent data due to concurrent access.

In a multi-process system, synchronization is necessary to ensure data consistency and integrity, and to avoid the risk of deadlocks and other synchronization problems. Process synchronization is an important aspect of modern operating systems, and it plays a crucial role in ensuring the correct and efficient functioning of multi-process systems.

On the basis of synchronization, processes are categorized as one of the following two types:

Independent Process: The execution of one process does not affect the execution of other processes.

Cooperative Process: A process that can affect or be affected by other processes executing in the system.

Advantages of Process Synchronization

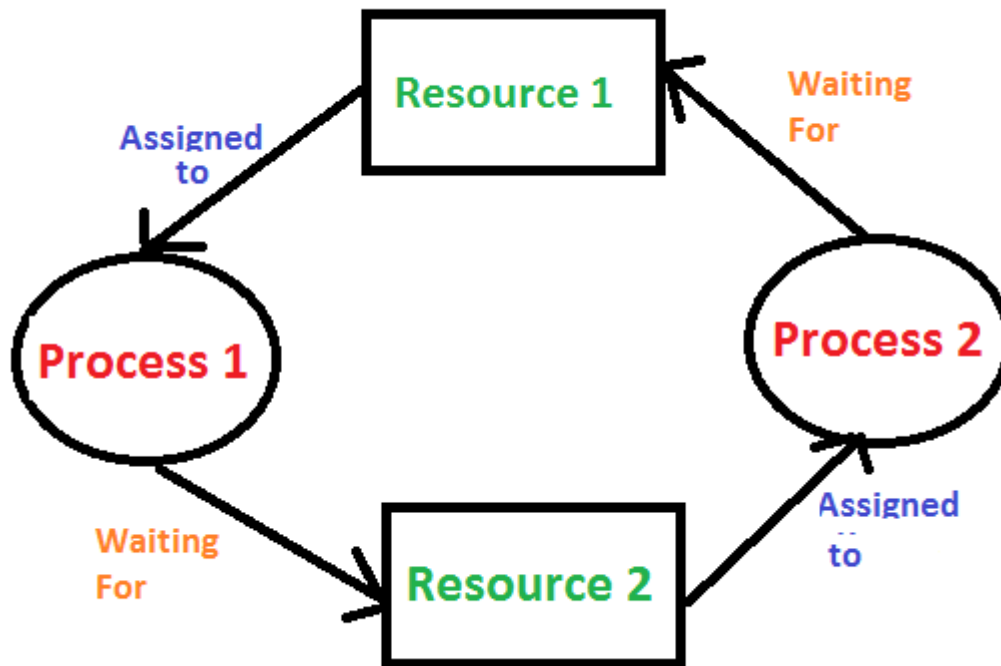
- Ensures data consistency and integrity
- Avoids race conditions
- Prevents inconsistent data due to concurrent access
- Supports efficient and effective use of shared resources

Disadvantages of Process Synchronization

- Adds overhead to the system
- This can lead to performance degradation
- Increases the complexity of the system
- Can cause deadlocks if not implemented properly.

Deadlocks.

A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.



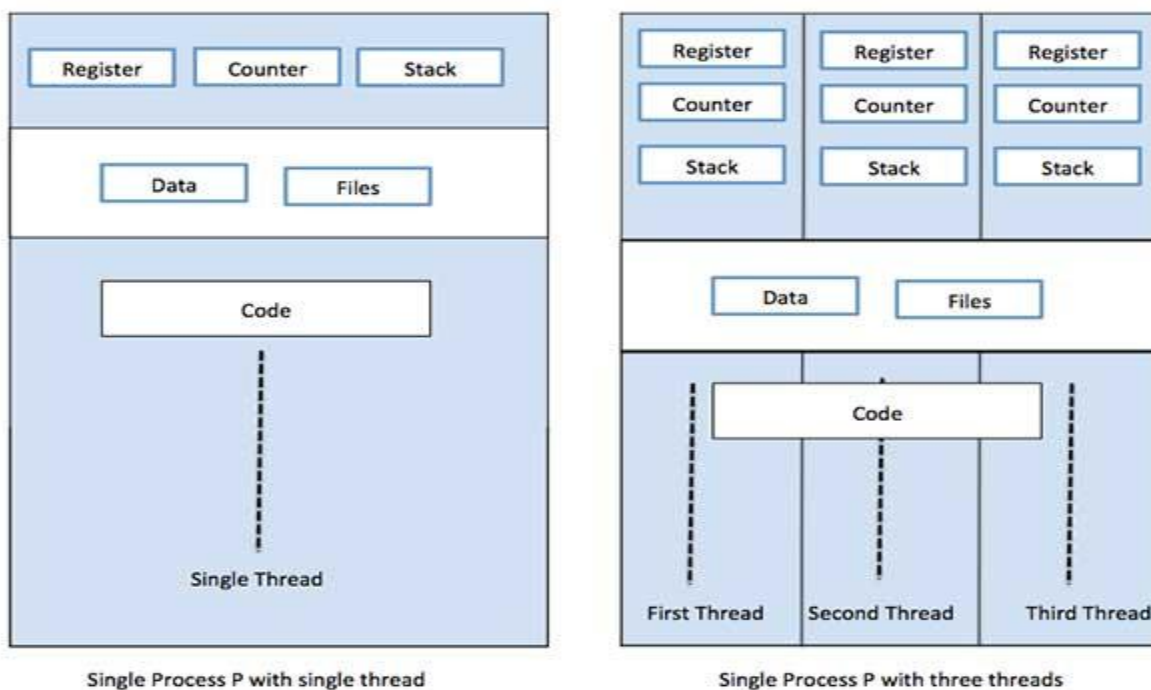
Consider an example when two trains are coming toward each other on the same track and there is only one track, none of the trains can move once they are in front of each other. A similar situation occurs in operating systems when there are two or more processes that hold some resources and wait for resources held by other(s). For example, in the below diagram, Process 1 is holding Resource 1 and waiting for resource 2 which is acquired by process 2, and process 2 is waiting for resource 1.

Thread Management

A thread is a flow of execution through the process code, with its own program counter that keeps track of which instruction to execute next, system registers which hold its current working variables, and a stack which contains the execution history.

A thread is also called a lightweight process. Threads provide a way to improve application performance through parallelism. Threads represent a software approach to improving performance of operating system by reducing the overhead thread is equivalent to a classical process.

Each thread belongs to exactly one process and no thread can exist outside a process. Each thread represents a separate flow of control. Threads have been successfully used in implementing network servers and web server. They also provide a suitable foundation for parallel execution of applications on shared memory multiprocessors. The following figure shows the working of a single-threaded and a multithreaded process.



Advantages of Thread

- Threads minimize the context switching time.
- Use of threads provides concurrency within a process.
- Efficient communication.
- It is more economical to create and context switch threads.
- Threads allow utilization of multiprocessor architectures to a greater scale and efficiency.

Difference between Process and Thread.

S.N.	Process	Thread
1	Process is heavy weight or resource intensive.	Thread is light weight, taking lesser resources than a process.
2	Process switching needs interaction with operating system.	Thread switching does not need to interact with operating system.
3	In multiple processing environments, each process executes the same code but has its own memory and file resources.	All threads can share same set of open files, child processes.
4	If one process is blocked, then no other process can execute until the first process is unblocked.	While one thread is blocked and waiting, a second thread in the same task can run.
5	Multiple processes without using threads use more resources.	Multiple threaded processes use fewer resources.
6	In multiple processes each process operates independently of the others.	One thread can read, write or change another thread's data.

System Calls.

A system call is a programmatic way in which a computer program requests a service from the kernel of the operating system it is executed on.

Services Provided by System Calls

- Process creation and management
- Main memory management
- File Access, Directory, and File system management
- Device handling(I/O)
- Protection
- Networking, etc.
 - **Process control:** end, abort, create, terminate, allocate, and free memory.
 - **File management:** create, open, close, delete, read files, etc.
 - Device management
 - Information maintenance
 - Communication

System Calls Advantages

- **Access to hardware resources:** System calls allow programs to access hardware resources such as disk drives, printers, and network devices.
- **Memory management:** System calls provide a way for programs to allocate and deallocate memory, as well as access memory-mapped hardware devices.
- **Process management:** System calls allow programs to create and terminate processes, as well as manage inter-process communication.
- **Security:** System calls provide a way for programs to access privileged resources, such as the ability to modify system settings or perform operations that require administrative permissions.
- **Standardization:** System calls provide a standardized interface for programs to interact with the operating system, ensuring consistency and compatibility across different hardware platforms and operating system versions.

File System.

A file system is a method a system software uses to store, organize, and manage files and directories on a storage device.

Advantages of File System

Organization: A file system allows files to be organized into directories and subdirectories, making it easier to manage and locate files.

Data protection: File systems often include features such as file and folder permissions, backup and restore, and error detection and correction, to protect data from loss or corruption.

Improved performance: A well-designed file system can improve the performance of reading and writing data by organizing it efficiently on disk.

Disadvantages of File System

Compatibility issues: Different file systems may not be compatible with each other, making it difficult to transfer data between different operating systems.

Disk space overhead: File systems may use some disk space to store metadata and other overhead information, reducing the amount of space available for user data.

Vulnerability: File systems can be vulnerable to data corruption, malware, and other security threats, which can compromise the stability and security of the system.

Module-04**RBT Level-L1, L2****Compiler and Linker:**

Compilers, Interpreters, and Linkers – Memory Allocation, Code Optimization, Linking and Relocation Concepts.

Introduction of Compiler:

The compiler is software that converts a program written in a high-level language (Source Language) to a low-level language (Object/Target/Machine Language/0, 1's).

Compiler is an intelligent program as compare to an assembler. Compiler verifies all types of limits, ranges, errors, etc.

Compiler program takes more time to run and it occupies huge amount of memory space.

The speed of compiler is slower than other system software. It takes time because it enters through the program and then does translation of the full program.

When compiler runs on same machine and produces machine code for the same machine on which it is running. Then it is called as self-compiler or resident compiler.

Compiler may run on one machine and produces the machine codes for other computer then in that case it is called as cross compiler.

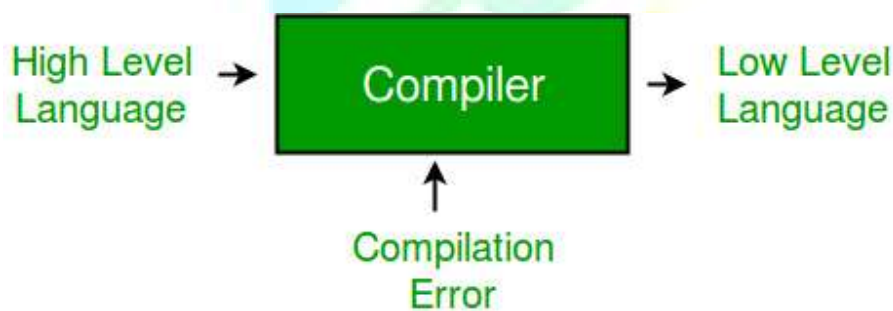
High-Level Programming Language

A high-level programming language is a language that has an abstraction of attributes of the computer. High-level programming is more convenient to the user in writing a program.

Low-Level Programming Language

A low-Level Programming language is a language that doesn't require programming ideas and concepts.

Stages of Compiler Design



Lexical Analysis: The first stage of compiler design is lexical analysis, also known as scanning. In this stage, the compiler reads the source code character by character and breaks it down into a series of tokens, such as keywords, identifiers, and operators. These tokens are then passed on to the next stage of the compilation process.

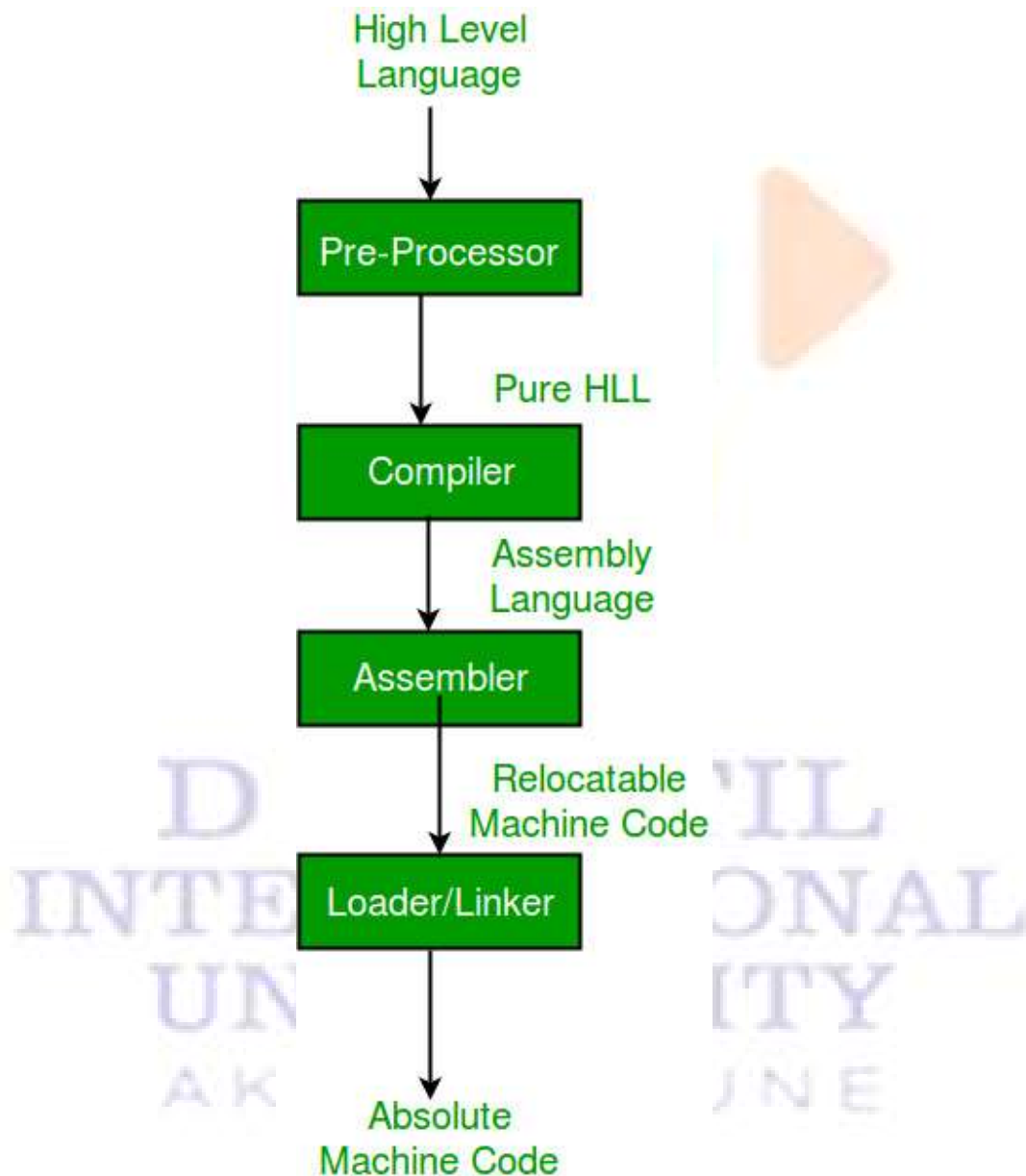
Syntax Analysis: The second stage of compiler design is syntax analysis, also known as parsing. In this stage, the compiler checks the syntax of the source code to ensure that it conforms to the rules of the programming language. The compiler builds a parse tree,

which is a hierarchical representation of the program's structure, and uses it to check for syntax errors.

Semantic Analysis: The third stage of compiler design is semantic analysis. In this stage, the compiler checks the meaning of the source code to ensure that it makes sense. The compiler performs type checking, which ensures that variables are used correctly and that operations are performed on compatible data types. The compiler also checks for other semantic errors, such as undeclared variables and incorrect function calls.

Code Generation: The fourth stage of compiler design is code generation. In this stage, the compiler translates the parse tree into machine code that can be executed by the computer. The code generated by the compiler must be efficient and optimized for the target platform.

Optimization: The final stage of compiler design is optimization. In this stage, the compiler analyzes the generated code and makes optimizations to improve its performance. The compiler may perform optimizations such as constant folding, loop unrolling, and function inlining.

Language Processing Systems:**Language Processing Systems:**

High-Level Language: If a program contains pre-processor directives such as #include or #define it is called HLL.

Pre-Processor: The pre-processor removes all the #include directives by including the files called file inclusion and all the #define directives using macro expansion. It performs file inclusion, augmentation, macro-processing, etc.

Assembly Language: It's neither in binary form nor high level. It is an intermediate state that is a combination of machine instructions and some other useful data needed for execution.

Assembler: For every platform (Hardware + OS) we will have an assembler. The output of the assembler is called an object file. It translates assembly language to machine code.

Interpreter: An interpreter converts high-level language into low-level machine language, just like a compiler. But they are different in the way they read the input.

Relocatable Machine Code: It can be loaded at any point and can be run. The address within the program will be in such a way that it will cooperate with the program movement.

Loader: Loader converts the relocatable code into absolute code and tries to run the program resulting in a running program or an error message (or sometimes both can happen).

Linker: linkers load a variety of object files into a single file to make it executable. Then loader loads it in memory and executes it.

Types of Compilers

There are mainly three types of compilers.

- ❖ Single Pass Compilers
- ❖ Two Pass Compilers
- ❖ Multipass Compilers

Single Pass Compiler

When all the phases of the compiler are present inside a single module, it is simply called a single-pass compiler. It performs the work of converting source code to machine code.

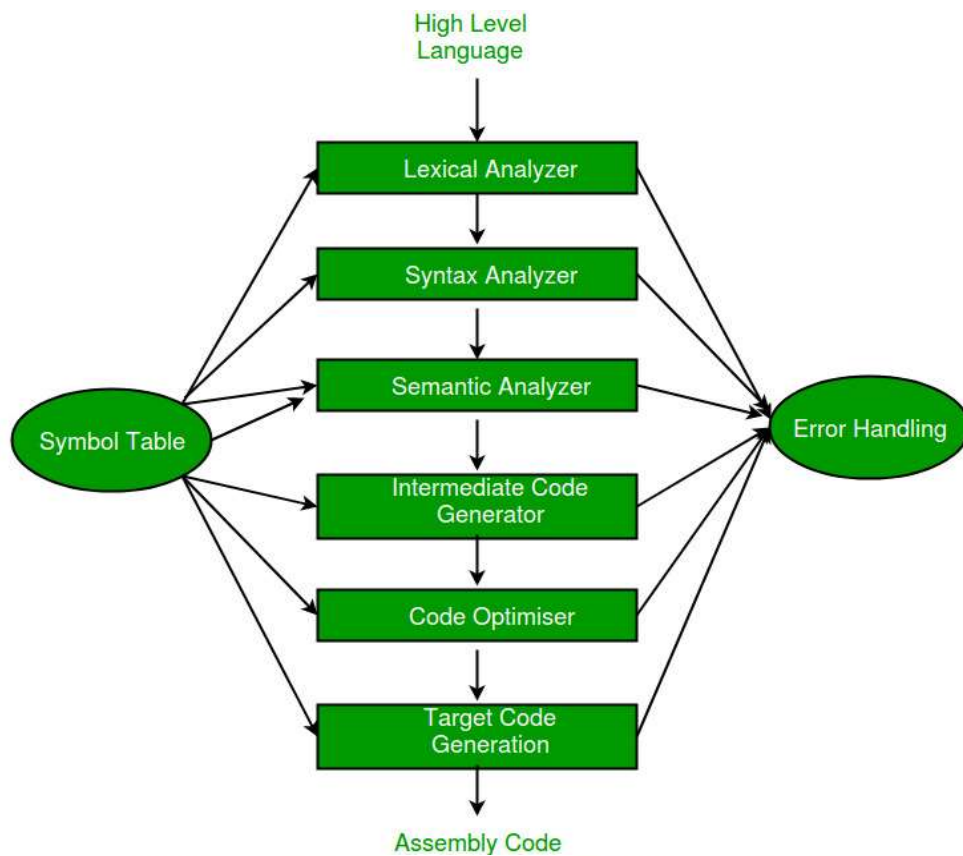
Two Pass Compiler

Two-pass compiler is a compiler in which the program is translated twice, once from the front end and the back from the back end known as Two Pass Compiler.

Multipass Compiler

When several intermediate codes are created in a program and a syntax tree is processed many times, it is called Multi pass Compiler. It breaks codes into smaller programs.

Phases of a Compiler



There are two major phases of compilation:

1) Analysis Phase

- Lexical Analyzer
- Syntax Analyzer
- Semantic Analyzer
- Intermediate Code Generator

2) Synthesis Phase

- Code Optimizer
- Code Generator

Analysis Phase: The analysis phase of system software involves gathering requirements for the system and studying business needs.

The **lexical analyzer** converts the High-level input program into a sequence of Tokens. Token is a sequence of characters.

The **Syntax analyzer** recognizes “sentences” in the program using the syntax of the language.

The **Semantic analyzer** checks the static semantics of each construct.

Intermediate Code Generator generates “abstract” code or target code.

Synthesis Phase: It takes the intermediate code generated by the front end of the compiler and converts it into machine code or assembly code, which can be executed by a computer.

Code Optimizer optimizes the abstract code, and the final **Code Generator** translates abstract intermediate code into specific machine instructions.

Operations of Compiler

These are some operations that are done by the compiler.

- ❖ It breaks source programs into smaller parts.
- ❖ It enables the creation of symbol tables and intermediate representations.
- ❖ It helps in code compilation and error detection.
- ❖ it saves all codes and variables.
- ❖ It analyses the full program and translates it.
- ❖ Convert source code to machine code.

Advantages of Compiler Design

Efficiency: Compiled programs are generally more efficient than interpreted programs because the machine code produced by the compiler is optimized for the specific hardware platform on which it will run.

Portability: Once a program is compiled, the resulting machine code can be run on any computer or device that has the appropriate hardware and operating system, making it highly portable.

Error Checking: Compilers perform comprehensive error checking during the compilation process, which can help catch syntax, semantic, and logical errors in the code before it is run.

Optimizations: Compilers can make various optimizations to the generated machine code, such as eliminating redundant instructions or rearranging code for better performance.

Disadvantages of Compiler Design

Longer Development Time: Developing a compiler is a complex and time-consuming process that requires a deep understanding of both the programming language and the target hardware platform.

Debugging Difficulties: Debugging compiled code can be more difficult than debugging interpreted code because the generated machine code may not be easy to read or understand.

Lack of Interactivity: Compiled programs are typically less interactive than interpreted programs because they must be compiled before they can be run, which can slow down the development and testing process.

Platform-Specific Code: If the compiler is designed to generate machine code for a specific hardware platform, the resulting code may not be portable to other platforms.

Interpreters.

Def: An interpreter is a type of computer program that directly executes instructions written in a programming or scripting language.



- ❖ All high-level languages need to be converted to machine code so that the computer can understand the program after taking the required inputs.
- ❖ The software by which the conversion of the high-level instructions is performed line-by-line to machine-level language, other than compiler and assembler, is known as **INTERPRETER**

Top Interpreters According to the Computer Languages

- ❖ **Python**-CPython, PyPy, Stackless Python, IronPython
- ❖ **Ruby**- YARV, Ruby MRI (CRuby)
- ❖ **JAVA**-HotSpot, OpenJ9, JRockIt
- ❖ **Kotlin**-JariKo

Need for Interpreters.

- ❖ If the source code is huge in size, then it might take hours to compile the source code, which will significantly increase the compilation duration.
- ❖ Here, the Interpreter plays its role. The interpreter can cut this huge compilation duration.
- ❖ So instead of waiting for the entire code, the interpreter translates a single line and executes it.

How does an interpreter work?

- ❖ An interpreter reads every line of code, translates it into machine code and then immediately executes it.
- ❖ The interpreter executes every source statement line by line during execution.
- ❖ An interpreter provides detailed error messages and supports interactive debugging.

Advantages of Interpreter

The following are the advantages of an interpreter:

Interactive debugging: Interpreters allow programmers to test their code interactively, meaning they can execute code one line at a time and see the results immediately. This makes it easier to debug code and identify errors quickly.

Ease of use: Interpreters typically have a simple and easy-to-use interface, making them accessible to novice programmers. Programmers can run their code without having to worry about the complexities of compilation and linking.

Portability: Interpreted code can be run on any platform that has an interpreter for the programming language used. This means that the same code can be run on different operating systems and hardware configurations without the need for modification.

Faster development: Interpreted languages allow programmers to write code more quickly because they can test their code immediately. This leads to faster development cycles and shorter time-to-market for software projects.

More detailed error messages: Interpreters can provide more detailed error messages than compilers because they analyze and execute code one line at a time. This can help programmers identify and fix errors more quickly.

Disadvantages of Interpreters

While interpreters offer several advantages, they also have some disadvantages, including:

Slower execution speed: Interpreted code can be slower to execute than compiled code because it is translated and executed line-by-line. This can result in slower program performance and is a concern for applications that require high performance.

Security risks: Interpreted languages can be more unprotected to security risks, such as injection attacks because they allow code to be executed dynamically. This poses a security risk if the interpreter is not properly secured.

Limited optimization: Interpreters are not able to optimize code to the same extent as compilers because they do not have the

opportunity to analyze the entire codebase before execution. This can result in less efficient code.

Limited scalability: Interpreters may have limitations when it comes to scaling, particularly when compared to compiled languages. This is because interpreted languages may require more resources to execute than compiled languages.

Higher memory usage: Interpreted languages may use more memory than compiled languages because they require more resources to store the code, the interpreter, and the runtime environment.

Difference between Compiler and Interpreter

Parameters	Compiler	Interpreter
Translation type	A compiler translates complete high-level programming code into machine code at once.	An interpreter translates one statement of programming code at a time into machine code.
Machine code	It stores the converted machine code from your source code program on the disk.	It never stores the machine code at all on the disk.
Running time	A compiler takes an enormous time to analyze source code. However, overall compiled programming code runs faster as compression to an interpreter.	An interpreter takes less time to analyze source code as compared to a compiler. However, overall interpreted programming code runs slower as compression to the compiler.
Program generation	The compiler generates an output of a program (in the	The interpreter doesn't generate a separate machine

	form of an exe file) that can run separately from the source code program.	code as an output program. So it checks the source code every time during the execution.
Execution	The process of program execution takes place separately from its compilation process. Program execution only takes place after the complete program is compiled.	The process of program execution is a part of interpretation steps, so it is done line-by-line simultaneously.
Memory requirement	A compiled program is generated into an intermediate object code, and it further required linking. So there is a requirement for more memory.	An interpreted program does not generate an intermediate code. So there is no requirement for extra memory.

Linker.

Def: Linker is a program in a system which helps to link object modules of a program into a single object file.

It performs the process of linking. Linkers are also called as link editors. Linking is a process of collecting and maintaining piece of code and data into a single file. Linker also links a particular module into system library.

Linking is performed at both compile time, when the source code is translated into machine code and load time, when the program is loaded into memory by the loader.

Linking is of two types:

1. Static Linking

2. Dynamic linking

Static Linking – It is performed during the compilation of source program. Linking is performed before execution in static linking.

Dynamic linking – Dynamic linking is performed during the run time. This linking is accomplished by placing the name of a shareable library in the executable image.

Advantages of Linker

There are several advantages of using a linker in compiler design:

Code Reuse: A linker allows code to be reused across multiple programs by linking in shared libraries, reducing the amount of code that needs to be written and maintained.

Smaller Executable Files: Dynamic linkers reduce the size of the executable file by linking libraries at runtime, rather than including them in the executable.

Reduced Memory Footprint: Dynamic linkers allow multiple programs to share the same library in memory, reducing the overall memory usage of the system.

Reduced Disk Space: With dynamic linking, the libraries only need to be stored on disk once, instead of being copied into the executable of each program that uses them.

Improved Security: Dynamic linkers enable the use of protected libraries, which can help prevent unauthorized access or modification of the library code.

Easier to Update Libraries: Dynamic linkers allow libraries to be updated or replaced without the need to relink the program, making it easier to fix bugs, add new features, or improve performance.

Portability: Linkers allow multiple object files generated by different compilers or written in different languages to be combined into a single executable file, allowing for more flexibility and portability in program

Disadvantages of Linker

Complexity: Linkers can be quite complex, especially when dealing with large and complex projects. This can make it difficult for developers to understand and troubleshoot issues that may arise.

Symbol resolution: Linkers must resolve symbols, which are names used in the source code that refer to memory locations. This can be a difficult and time-consuming process, especially when dealing with multiple object files and libraries.

Compatibility issues: Linkers must be able to work with a variety of object file formats and libraries, which can be challenging. Incompatible object files or libraries can cause errors or crashes during the linking process.

Performance: Linkers can be resource-intensive and may take a long time to process large projects. This can be a problem for developers working on large-scale projects or for embedded systems with limited resources.

Security: Linkers can also be a potential security risk if not properly implemented. Insecure linking can lead to vulnerabilities that can be exploited by malicious actors.

Dependency management: Linkers also require proper dependency management. If the dependencies are not managed properly, the final binary may not run properly or may not run at all.

Applications of Linker

There are several key applications of linkers in compiler design, including:

Combining object files: The primary function of a linker is to combine multiple object files into a single executable. This allows for the efficient management and distribution of large software projects, as multiple object files can be combined into a single executable that can be run on a target platform.

Resolving symbol references: Linkers also resolve symbol references between object files. A symbol is a variable or function that is defined in one object file and used in another. The linker ensures that all symbol references are properly resolved, so that the final executable can run correctly.

Dynamic linking: Linkers also support dynamic linking, which allows a program to call functions or access variables that are defined in a separate shared library. This allows for more efficient memory usage, as multiple programs can share the same library.

Library management: Linkers also play an important role in library management. A library is a collection of object files that can be linked into a program to provide additional functionality. Linkers allow developers to easily link libraries into their programs, making it easier to add new functionality to a project.

Modularity: Linkers also allow for greater code modularity and reusability. By breaking a software program into multiple object files that can be linked together, developers can create more modular and reusable code.

Memory Allocation

What is Memory Management?

Memory management is a method of subdividing the memory among different processes. The main aim of memory management is to achieve efficient utilization of memory.

Why Memory Management is Required?

- ❖ Allocate and de-allocate memory before and after process execution.
- ❖ To keep track of used memory space by processes.
- ❖ To minimize fragmentation issues.
- ❖ To proper utilization of main memory.
- ❖ To maintain data integrity while executing of process.

Advantages of Memory Management

- ❖ It is a simple management approach

Disadvantages of Memory Management

- ❖ It does not support multiprogramming
- ❖ Memory is wasted

Code Optimization.

Code optimization is a program transformation technique that aims to improve code by reducing resource consumption while maintaining high performance. This is done by replacing high-level general programming structures with highly efficient low-level programming codes.

Advantages of Code Optimization:

Improved performance: Code optimization can result in code that executes faster and uses fewer resources, leading to improved performance.

Reduction in code size: Code optimization can help reduce the size of the generated code, making it easier to distribute and deploy.

Increased portability: Code optimization can result in code that is more portable across different platforms, making it easier to target a wider range of hardware and software.

Reduced power consumption: Code optimization can lead to code that consumes less power, making it more energy-efficient.

Improved maintainability: Code optimization can result in code that is easier to understand and maintain, reducing the cost of software maintenance.

Disadvantages of Code Optimization:

Increased compilation time: Code optimization can significantly increase the compilation time, which can be a significant drawback when developing large software systems.

Increased complexity: Code optimization can result in more complex code, making it harder to understand and debug.

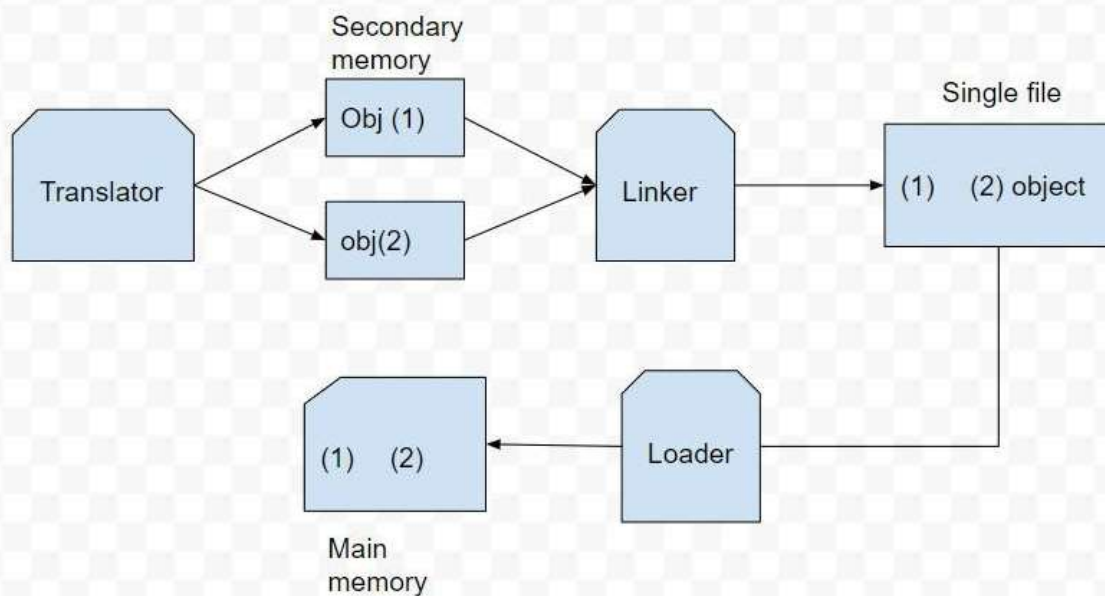
Potential for introducing bugs: Code optimization can introduce bugs into the code if not done carefully, leading to unexpected behavior and errors.

Difficulty in assessing the effectiveness: It can be difficult to determine the effectiveness of code optimization, making it hard to justify the time and resources spent on the process.

Linking and Relocation Concepts.

Linker.

- ❖ Linker is a program in a system which helps to link object modules of a program into a single object file.
- ❖ It performs the process of linking. Linkers are also called as link editors.
- ❖ Linking is a process of collecting and maintaining piece of code and data into a single file.



Linking is of two types

- ❖ Static Linking
- ❖ Dynamic linking

Static Linking- It is performed during the compilation of source program.

Static linker performs two major tasks:

- ❖ **Symbol resolution** – It associates each symbol reference with exactly one symbol definition. Every symbol has a predefined task.
- ❖ **Relocation** – It relocates code and data section and modifies the symbol references to the relocated memory locations.

Dynamic linking – Dynamic linking is performed during the run time.

Features :

Symbol resolution: The linker resolves symbols, such as function and variable names, across different object files and libraries.

Relocation: The linker performs relocation, adjusting the addresses of symbols within object files and libraries to match the final address space of the executable program.

Optimization: The linker can perform optimization, such as dead code elimination and function inlining, to improve the performance and size of the executable program.

Library management: The linker can manage libraries, linking in only the required functions and removing unused code to minimize the size of the executable.

Debugging information: The linker can include debugging information in the executable program, making it easier to debug and analyze during development.

Cross-platform support: The linker can generate executable programs for different platforms, including different architectures and operating systems.

Incremental linking: The linker can perform incremental linking, allowing changes to be made to individual object files without needing to rebuild the entire executable program.

Versioning: The linker can support versioning of shared libraries, allowing multiple versions of a library to coexist and preventing compatibility issues.

Link-time code generation: The linker can perform link-time code generation, allowing code to be generated during the linking process rather than at compile time.

Linker scripts: The linker can use linker scripts, which are configuration files that specify how object files and libraries should be linked together. Linker scripts can also be used to specify the layout of the executable program's memory.

Advantages of Linker

There are several advantages of using a linker in compiler design:

Code Reuse: A linker allows code to be reused across multiple programs by linking in shared libraries, reducing the amount of code that needs to be written and maintained.

Smaller Executable Files: Dynamic linkers reduce the size of the executable file by linking libraries at runtime, rather than including them in the executable.

Reduced Memory Footprint: Dynamic linkers allow multiple programs to share the same library in memory, reducing the overall memory usage of the system.

Reduced Disk Space: With dynamic linking, the libraries only need to be stored on disk once, instead of being copied into the executable of each program that uses them.

Improved Security: Dynamic linkers enable the use of protected libraries, which can help prevent unauthorized access or modification of the library code.

Easier to Update Libraries: Dynamic linkers allow libraries to be updated or replaced without the need to relink the program, making it easier to fix bugs, add new features, or improve performance.

Portability: Linkers allow multiple object files generated by different compilers or written in different languages to be combined into a single executable file, allowing for more flexibility and portability in program

Disadvantages of Linker

Complexity: Linkers can be quite complex, especially when dealing with large and complex projects. This can make it difficult for developers to understand and troubleshoot issues that may arise.

Symbol resolution: Linkers must resolve symbols, which are names used in the source code that refer to memory locations. This can be a difficult and time-consuming process, especially when dealing with multiple object files and libraries.

Compatibility issues: Linkers must be able to work with a variety of object file formats and libraries, which can be challenging. Incompatible object files or libraries can cause errors or crashes during the linking process.

Performance: Linkers can be resource-intensive and may take a long time to process large projects. This can be a problem for developers working on large-scale projects or for embedded systems with limited resources.

Security: Linkers can also be a potential security risk if not properly implemented. Insecure linking can lead to vulnerabilities that can be exploited by malicious actors.

Dependency management: Linkers also require proper dependency management. If the dependencies are not managed properly, the final binary may not run properly or may not run at all.

Applications of Linker

There are several key applications of linkers in compiler design, including:

Combining object files: The primary function of a linker is to combine multiple object files into a single executable. This allows for the efficient management and distribution of large software projects, as multiple object files can be combined into a single executable that can be run on a target platform.

Resolving symbol references: Linkers also resolve symbol references between object files. A symbol is a variable or function that is defined in one object file and used in another. The linker ensures that all symbol references are properly resolved, so that the final executable can run correctly.

Dynamic linking: Linkers also support dynamic linking, which allows a program to call functions or access variables that are defined in a separate shared library. This allows for more efficient memory usage, as multiple programs can share the same library.

Library management: Linkers also play an important role in library management. A library is a collection of object files that can be linked into a program to provide additional functionality. Linkers allow developers to easily link libraries into their programs, making it easier to add new functionality to a project.

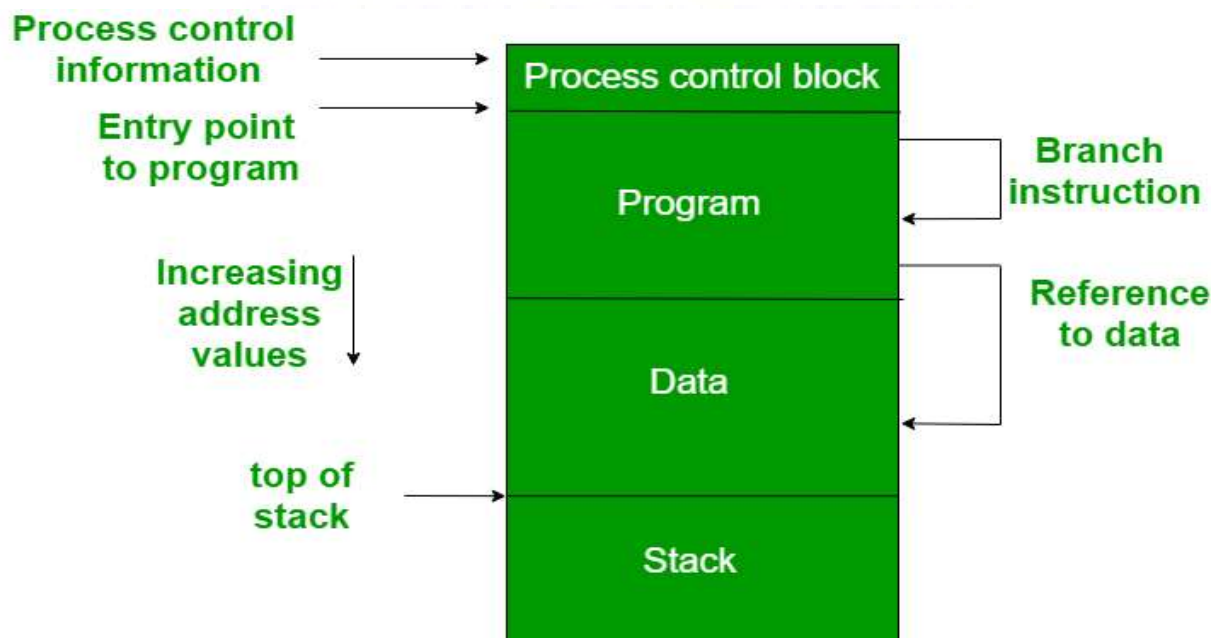
Modularity: Linkers also allow for greater code modularity and reusability. By breaking a software program into multiple object files that can be linked together, developers can create more modular and reusable code.

Relocation Concepts.

Relocation is the process of updating the addresses used in a program's address-sensitive instructions. This process is necessary to help execute the program from a designated area of memory.

The available memory is generally shared among a number of processes in a multiprogramming system, so it is not possible to know in advance which other programs will be resident in main memory at the time of execution of this program. Swapping the active processes in and out of the main memory enables the operating system to have a larger pool of ready-to-execute process.

When a program gets swapped out to a disk memory, then it is not always possible that when it is swapped back into main memory then it occupies the previous memory location, since the location may still be occupied by another process. We may need to **relocate** the process to a different area of memory. Thus there is a possibility that program may be moved in main memory due to swapping.



The figure depicts a process image. The process image is occupying a continuous region of main memory. The operating system will need to know many things including the location of process control information, the execution stack, and the code entry. Within a program, there are memory references in various instructions and these are called logical addresses.

After loading of the program into main memory, the processor and the operating system must be able to translate logical addresses into physical addresses. Branch instructions contain the address of the next instruction to be executed. Data reference instructions contain the address of byte or word of data referenced.



Module-05**RBT Level-L1, L2****Assembler:**

Basic function, Machine dependent and independent assembler features, Assembler design options: Two-pass, one-pass and multi-pass assembler design.

Assembler is a program for converting instructions written in low-level assembly code into relocatable machine code and generating along information for the loader.

OR

Assembler can be defined as a program that translates an assembly language program into a machine language program.



It generates instructions by evaluating the mnemonics (symbols) in operation field and find the value of symbol and literals to produce machine code. Now, if assembler do all this work in one scan then it is called single pass assembler, otherwise if it does in multiple scans then called multiple pass assembler. Here assembler divide these tasks in two passes:

Pass-1:

- Define symbols and literals and remember them in symbol table and literal table respectively.
- Keep track of location counter
- Process pseudo-operations

- Defines program that assigns the memory addresses to the variables and translates the source code into machine code

Pass-2:

- Generate object code by converting symbolic op-code into respective numeric op-code
- Generate data for literals and look for values of symbols
- Defines program which reads the source code two times
- It reads the source code and translates the code into object code.

Machine dependent and independent assembler features

Machine dependent assembler: A machine-dependent assembler is a program that converts assembly language into machine code, allowing for direct communication with a computer's hardware.

Machine dependent assembler features.

- **Instruction formats**
- **Addressing modes**
- **Program relocation**

Instruction format is the structure of machine language instructions for the processor to execute. The instruction formats are a sequence of bits (0 and 1). These bits, when grouped, are known as fields. Each field of the machine provides specific information to the CPU related to the operation and location of the data. The instruction format also defines the layout of the bits for an instruction.

Addressing modes The term addressing modes refers to the way in which the operand of an instruction is specified. The addressing mode specifies a rule for interpreting or modifying the address field of the instruction before the operand is actually executed.

Program relocation A Relocatable program is one that can be read into memory at any address and executed without modification. Using this information, it is possible to relocate the program to execute from a storage area other than the one designated for it at the time of its coding or translation.

Machine independent assembler features

These are the features which do not depend on the architecture of the machine. Such features are more related to software than to machine architecture. These are:

- Literals
- Symbol defining statements
- Expressions
- Program blocks
- Control sections

Literals:

- It is easy for a programmer to write the value of a constant operand as part of the instruction that uses it.
- This avoids defining the constant elsewhere in the program and making a label for it. Such an operand is called a literal because the value is stated literally in the instruction.

Symbol-defining statements:

These are a machine-independent feature of system software that allows programmers to define symbols and specify their values.

Expressions:

Assemblers also allow use of expressions in place of operands in the instruction. Each such expression must be evaluated to generate a single operand value or address. Assemblers generally arithmetic expressions formed according to the normal rules using arithmetic operators +, -, *, /. Division is usually defined to produce an integer result.

Program blocks:

- Program blocks allow the generated machine instructions and data to appear in the object program in a different order by Separating blocks for storing code, data, stack, and larger data block.
- Program blocks refer to segments of code that are rearranged within a single object program unit.

Control sections

A control section is a part of the program that maintains its identity after assembly; each control section can be loaded and relocated independently of the others.

Different control sections are most often used for subroutines or other logical subdivisions. The programmer can assemble, load, and manipulate each of these control sections separately.

Because of this, there should be some means for linking control sections together. For example, instructions in one control section may refer to the data or instructions of other control sections.

Since control sections are independently loaded and relocated, the assembler is unable to process these references in the usual way. Such references between different control sections are called external references.

Assembler design options

There are Three design options or the assembler.

- **One-pass assemblers**
- **Two-pass assemblers**
- **Multi-pass assemblers**

One pass assembler: Convert assembly code to machine code in one pass.



The main problem in designing the assembler using single pass was to resolve forward references. We can avoid to some extent the forward references by:

- Eliminating forward reference to data items, by defining all the storage reservation statements at the beginning of the program rather at the end.
- Unfortunately, forward reference to labels on the instructions cannot be avoided. (forward jumping)
- To provide some provision for handling forward references by prohibiting forward references to data items.

One-pass assemblers can be classified into two types:

Load-and-go assemblers: Produce object code directly in memory for immediate execution.

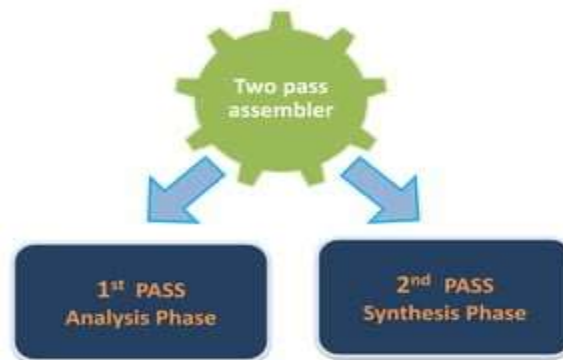
One pass assemblers generating object code for later execution: Produce the usual kind of object program for later execution.

Here are some other features of assembly language:

- Mnemonic instructions
- Direct access to hardware
- Low-level abstraction

- Efficient use of resources
- Full control over program flow
- Direct access to memory
- Better control over CPU

Two-pass assemblers: Generate the symbol table in pass 1 and machine code in pass 2.



An assembler has two main phases:

Analysis phase: Validates the code's syntax, checks for errors, and creates a symbol table

Synthesis phase: Uses information from the Analysis phase to convert the assembly language instructions into machine code

Multipass Assembler: A multi-pass assembler, also known as a two-pass assembler, scans the code multiple times, stores values in the opcode table and symbol table, and then converts the code to machine. The first pass defines symbols and literals, and the second pass generates an object program. The assembler has several other features, including:

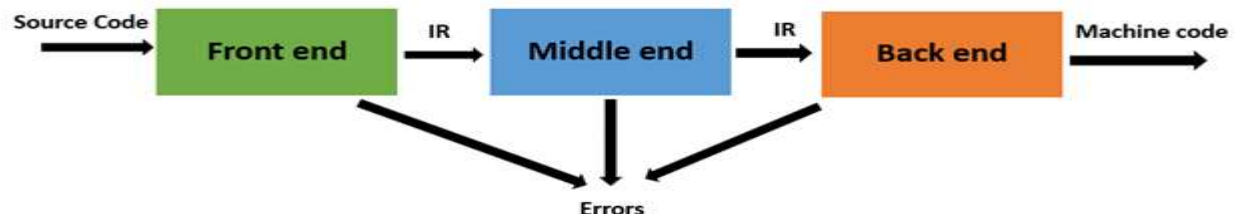
Symbol table: Contains labels and their values

Parsing: Moves in program lines to pull out op-codes and operands

Data values: Generates data values defined by WORD or BYTE

Processing assembler directives: Processes assembler directives not done in Pass 1

Optimization: Has advantages for optimization



- A multipass assembler can make as many passes as needed to process the definition of symbols.
- For a forward reference in symbol definition, we store in the SYMTAB:
 - The symbol name
 - The defining expression
 - The number of undefined symbols in the defining expression
- The undefined symbol associated with a list of symbols depend on this undefined symbol.
- When a symbol is defined, we can recursively evaluate the symbol expressions depending on the newly defined symbol.

Difference Between Single Pass and Multi Pass assembler's design

Single Pass	Multi Pass
Passes through each compilation step only once, instantly converting each token into the required machine code.	Passes the source code through the compilation process multiple times.
Faster than multi-pass compiler	It is comparatively slower as it

	goes through several passes and writes intermediate code in each pass.
It has a limited scope.	It has a greater scope.
It is also known as a narrow compiler.	It is also called a wide compiler.
here is no code optimization, and the code generation is less efficient.	Better code generation and optimization.
There is no intermediate code generation.	The intermediate code is generated during the process.
It takes less time to compile.	It takes more time to compile.

D Y PATIL
INTERNATIONAL
UNIVERSITY
AKURDI PUNE



D Y PATIL INTERNATIONAL UNIVERSITY

AKURDI, PUNE-44, MAHARASHTRA, INDIA

Website: www.dypiu.ac.in, E-mail: info@dypiu.com

(Established under Maharashtra Act No. LXIII of 2017)

School of Computer Science, Engineering and Applications

Systems Software _CSE 2103_ Question Bank

S.No.	Questions	CO	RBT
Module-01			
01	With neat block diagram Explain Structure of a Computer System	1	L1,L2
02	Define operating system? Mentation function of operating system.	1	L1,L2
03	List out the Objectives of Operating Systems	1	L1,L2
04	List and Explain types of Operating Systems	1	L1,L2
05	Explain the how to Choose the Operating System	1	L1,L2
Module-02			
01	Define kernel and explain resources of kernel.	2	L1,L2
02	Write short note on Shell	2	L1,L2
03	Define shell scripting and explain Why use shell programming.	2	L1,L2
04	Explain the followings a) gedit b) nano editor c) Vi editor d) GUI	2	L1,L2
05	Explain the echo command and options Available in `echo` command in Linux	2	L1,L2
06	Define variable? List out the rules for variable definition, Explain the types of variables	2	L1,L2
07	What is Shell Variable and Shell Scripting?	2	L1,L2
08	Write short note on read User Input	2	L1,L2
09	With example Explain the followings • if • if-else-if • if-elif-else-if • nested if-else	2	L1,L2
10	With example Explain the followings • for • while • do	2	L1,L2
11	Define Functions and Explain Basic Structure of a Function in Shell Scripting	2	L1,L2
12	Write short note on • Pass Parameters to a Function • Functions with Passing Arguments		
Module-03			
01	With example explain process of Operating system	3	L1,L2

02	With neat block diagram, Explain Process structure or Process Life Cycle.	3	L1,L2
03	Write short note on Process Scheduling	3	L1,L2
04	List difference Between Preemptive and Non-Preemptive Scheduling	3	L1,L2
05	With neat block diagram, Explain Process Scheduling Queues	3	L1,L2
06	Distinguish between Long-Term Scheduler, Short-Term Scheduler and Medium-Term Scheduler	3	L1,L2
07	Write short note on First Come First Serve (FCFS) Scheduling Algorithm.	3	L1,L2
08	Write short note on Shortest Job First (SJF) Scheduling Algorithm.	3	L1,L2
09	Write short note on Longest Job First (LJF) Scheduling Algorithm.	3	L1,L2
10	Write short note on Priority Scheduling Scheduling Algorithm.	3	L1, L2
11	Write short note on round robin Scheduling Algorithm.	3	L1, L2
12	Write short note on 1) Shortest Remaining Time First 2) Longest Remaining Time First 3) Highest Response Ratio Next 4) Multiple Queue Scheduling Algorithm.	3	L1, L2
13	Write short note on Deadlocks.	3	L1, L2
14	Distinguish between Process Thread	3	L1, L2
15	Write short note on System Calls and File System.	3	L1, L2
Module-04			
01	Define compiler? With neat block diagram, Explain Stages of Compiler Design	4	L1, L2
02	With neat block diagram, Explain Language Processing Systems	4	L1, L2
03	Explain Types of Compilers	4	L1, L2
04	With neat block diagram, Explain Phases of a Compiler	4	L1, L2
05	Define Code Optimizer? Mention its Operations, Advantages & Disadvantages of Compiler Design	4	L1, L2
06	Define Interpreters? Mention its Advantages & Disadvantages of Interpreters	4	L1, L2
07	List difference between Compiler and Interpreter	4	L1, L2
08	Define Linker? Mention its Advantages & Disadvantages of Linker	4	L1, L2
09	Write short note on Linking and Relocation Concepts.	4	L1, L2
Module-05			
01	Define Assembler? With neat block diagram, Explain operation of assembler	5	L1, L2
02	Define Machine dependent assembler and mention its features.	5	L1, L2
03	Define Machine independent assembler and mention its features.	5	L1, L2
04	Briefly explain the Assembler design	5	L1, L2
05	List difference Between Single Pass and Multi Pass assembler's design	5	L1, L2